

Middlesex University

Department of Computer Science

**Automating the Purple Team Lifecycle:
An Integrated Deterministic AI Framework
for
Vulnerability Discovery, Prioritisation, and
Remediation Guidance**

A dissertation submitted in partial fulfilment of the requirements
for the degree of

**Bachelor of Science (Honours)
Cyber Security and Digital Forensics**

CST3590 — Individual Project (Dissertation)

Student Name: Umar, Majid Khan Faiz
Student ID: M01031005
Supervisor: David Neilson

Declaration

I declare that this dissertation is my own work and has not been submitted in whole or in part for any other degree or professional qualification. All sources of information have been acknowledged and referenced in accordance with IEEE referencing standards. I confirm that this work complies with Middlesex University's regulations on academic integrity and the appropriate use of artificial intelligence tools.

Name: Umar, Majid Faiz

Signature:  **Date:** 24 April 2026

Abstract

Security tooling for web vulnerability assessment has matured independently across several specialised domains. Automated scanners produce structured, reproducible evidence; scoring systems rank findings by severity; large language models summarise vulnerabilities and propose remediation steps; and validation research constrains AI output reliability. Few systems integrate these capabilities into a single pipeline that closes the scan-remediate-rescan loop while grounding AI analysis in deterministic scanner evidence rather than probabilistic inference. The absence of this integration leaves a structural gap between evidence generation and validated remediation guidance, requiring analysts to bridge fragmented tools manually at each stage of the assessment lifecycle.

PurpleTeam Suite was designed and implemented to address this gap. The system is an Electron desktop application implementing a ten-stage pipeline: Nmap-based network scanning across two progressive phases, deterministic OWASP Top 10 classification via a weighted keyword mapper, exploitability-aware contextual scoring with four multipliers (port exposure, service exposure, authentication weakness, TLS absence), Gemini-powered AI analysis under structured JSON schema enforcement and constrained temperature settings, a hallucination guard that cross-validates AI output against deterministic scan evidence, a delta comparison module supporting iterative scan-remediate-rescan cycles, and exportable PDF reporting. A runtime feature toggle system enables controlled per-component ablation without codebase modification.

The evaluation framework comprised five phases executed across five authorised targets: Mutillidae II via Docker, a purpose-built controlled localhost testbed, scanme.nmap.org, testasp.vulnweb.com and demo.testfire.net. Phase 1 established baseline consistency and reproducibility; Phase 2 compared flat versus contextual severity scoring; Phase 3 performed per-component ablation on Mutillidae; Phase 4 demonstrated the iterative lifecycle on the controlled testbed; and Phase 5 accumulated longitudinal hallucination guard metrics across all targets.

Contextual weighting produced a 69.2% amplification in severity deductions on Mutillidae relative to flat scoring. Ablation testing identified contextual weighting as the highest-impact pipeline component, with a 74-point delta when disabled. The scan-remediate-rescan cycle on the controlled testbed produced a 57-point score improvement (3 to 60) and a 53% reduction in total findings (15 to 7). Zero fabricated CVE identifiers were recorded across all 11 AI-guarded scans. Three of the five evaluation targets fell below the 70-point trust score threshold, a result that reflects genuine AI classification inconsistency driven by target complexity and technology stack rather than a systematic

deficiency in the hallucination guard. Trust scores ranged from 0 on the Mutillidae ablation configuration (expected, with the OWASP mapper disabled) to approximately 85 on `testasp.vulnweb.com`.

The project demonstrates that a deterministic-first integration with constrained AI assistance is operationally viable, and that hallucination mitigation can be grounded in scanner evidence rather than surface plausibility assessment. Limitations include single-provider LLM dependency on Google Gemini, a network-layer scanning scope that structurally bounds OWASP coverage to 4 of 10 categories (with architectural categories A04, A08 and A09 remaining beyond automated reach), researcher-calibrated rather than empirically validated contextual multipliers, and an evaluation corpus of five authorised targets that limits generalisability. These constraints point toward concrete future directions: multi-provider LLM comparison, empirical multiplier calibration using exploit-prediction data, and coverage extension through application-layer scanner integration.

Acknowledgements

I transferred to Middlesex University for personal reasons and felt at the time that I had taken a step back, arriving quite out of place, with low confidence in my own abilities and genuinely lost in what I should be doing with my situation. It was David Neilson who changed that. He could see that I was eager, gave me the autonomy to go at whatever pace I was capable of, and allowed that to develop naturally rather than directing it. Over the two years I have known him, this not only produced a genuine love for the subjects he taught but also gave me the space to grow into a version of myself that felt settled and capable, and David is someone I have been able to be brutally honest with in conversations that I have not been able to have with most other people. He is someone who would take time out of his own day to talk through issues that genuinely supported me, not only acting as a sounding board on which masters programmes I should apply to but actively pushing me to aim higher than I would have on my own. He supported my decision to start and run the Cyber Security Society, something that was incredibly difficult for someone who at the time struggled to talk in front of others and was deeply introverted, and watching that grow to around fifty members, attending hackathons and events, gave me a confidence I did not have before. The outcome of all of that is something I still cannot quite believe but am so utterly grateful for, which is that I now hold an offer for an MSc at King's College London, and I am certain I would not be in that position, or in the mental state to take it seriously, without the confidence this relationship helped me build.

Khalid, my younger brother, has been a constant across my time at university and through the side projects that ran alongside it. We are very different people. I have always been someone who plans everything out, sticks to the plan, and struggles when things do not go the way I expected, and Khalid is the opposite of that in ways I genuinely admire. He has faced circumstances across his education and his personal life that would have justified giving up entirely, situations that I honestly do not think I could have handled myself, and he has come out of every one of them calmer, more grounded and better for it rather than using any of it as an excuse. What I appreciate most about him is that he approaches life with a lightness that has consistently balanced me out when I have needed it, and during the periods where I have come close to burning out he has been the person I could simply stop with, take a breath with, and step back from whatever I was too deep in. This is a relationship where we both, without thought, support each other and look out for each other, and I am grateful for it.

Contents

Declaration	i
Abstract	ii
Acknowledgements	iv
1 Introduction	1
1.1 Background & Context	1
1.2 Research Questions	2
1.3 Aims	3
1.3.1 Objectives	3
1.3.2 Deliverables	3
2 Literature Review	5
2.1 Introduction	5
2.2 Detection Maturity and the Limits of Automated Scanning	5
2.3 Risk Scoring Models	6
2.4 AI-Assisted Classification	7
2.5 Remediation Automation	8
2.6 Hallucination and LLM Reliability	9
2.7 Purple Team Lifecycle Integration Gaps	10
2.8 Summary and Transition to Methodology	11
3 Methodology	13
3.1 Research Approach	13
3.2 System Design	14
3.2.1 Requirements	14
3.2.2 Pipeline Architecture	15
3.2.3 Scanning and Detection	16
3.2.4 Classification and Scoring	17
3.2.5 AI Integration and Hallucination Control	17
3.2.6 Lifecycle Support	18
3.2.7 Implementation Environment	20
3.3 Evaluation Framework	23
3.4 Limitations and Ethical Considerations	25
3.5 Summary	25

4	Analysis and Results	26
4.1	Evaluation Design Overview	26
4.2	Phase 1: Baseline Consistency and Reproducibility	27
4.3	Phase 2: Contextual Weighting Comparative Analysis	30
4.4	Phase 3: Ablation Testing	34
4.5	Phase 4: Lifecycle Delta Demonstration	35
4.6	Phase 5: Longitudinal Hallucination Reliability	37
4.7	Tool Capability Comparison	39
4.8	Summary	39
5	Conclusions	41
5.1	Introduction	41
5.2	Answering the Research Questions	42
5.3	Contributions	43
5.4	Limitations	43
5.5	Recommendations and Future Work	44
	Bibliography	45
	Appendices	49
A	Supervisor Meeting Log	50
B	Repository Information and Statistics	52
C	Project Schedule	54
D	Implementation Environment Details	55

List of Figures

3.1	PurpleTeam Suite pipeline architecture. Each stage is implemented as an independent module. The dashed feedback arrow represents the iterative scan-remediate-rescan lifecycle supported by the delta comparison system.	16
3.2	Scan target input interface with allowlist validation. The six-step progress stepper is visible prior to scan execution.	17
3.3	PurpleTeam Suite dashboard displaying security score with exploitability-aware weighting, OWASP Top 10 coverage matrix and prioritised vulnerability table.	18
3.4	AI-generated vulnerability analysis showing plain-English summary, severity justification and structured remediation steps. The hallucination guard trust indicators are displayed alongside each finding.	19
3.5	Delta comparison outputs in PurpleTeam Suite, showing both the application report interface and the exported report evidence used in lifecycle evaluation.	20
4.1	Severity deduction scores across all five baseline scans, illustrating the range from 50 (testasp.vulnweb.com) to 181 (Mutillidae).	28
4.2	Severity distribution by target, showing the breakdown of Critical, High, Medium, Low and Informational findings across all five baseline scans. .	28
4.3	OWASP Top 10 category heatmap across all five baseline scans, with cell values indicating the number of vulnerabilities mapped to each category.	29
4.4	Comparison of flat (weighting OFF) versus contextual (weighting ON) severity deductions on the Mutillidae target, showing the 69.2% amplification produced by the contextual multiplier system.	31
4.5	Per-vulnerability contextual multiplier values for the Mutillidae baseline scan, showing port exposure, service exposure, authentication weakness and combined multiplier across all 26 findings.	32
4.6	Adjusted deduction scores for each vulnerability in the Mutillidae baseline scan, corresponding to the multiplier values in Figure 4.5.	33
4.7	Ablation test severity deductions on the Mutillidae target, comparing the full pipeline against each single-component removal configuration. .	35
4.8	Lifecycle delta comparison between the baseline and remediated scans on the controlled testbed, showing the score improvement from 3 to 60 and the reduction from 15 to 7 total vulnerabilities.	36
4.9	Trust scores across all evaluation scans, with the 70-point threshold indicated. Only scanme.nmap.org and testasp.vulnweb.com consistently exceed the threshold.	38

4.10	Hallucination risk flag breakdown per scan, showing stacked Low, Medium and High risk flags. Mutillidae baseline exhibits the highest total flag count at approximately 18 flags.	38
B.1	PurpleTeam Suite running on Kali Linux.	52
B.2	PurpleTeam Suite running on macOS.	53
C.1	Planned versus actual project schedule.	54

List of Tables

3.1	Requirements traceability matrix mapping proposal objectives to system modules and evaluation phases.	14
3.2	Core technology stack and selection rationale.	21
3.3	Key library dependencies and pipeline integration points.	22
3.4	Five-phase evaluation framework with target configurations, methods and key output metrics.	24
4.1	Evaluation targets and their roles within the five-phase framework. . .	26
4.2	Phase 1 baseline scan results across all five evaluation targets (all feature toggles enabled).	27
4.3	OWASP Top 10 category distribution across baseline scans.	27
4.4	Phase 2 contextual weighting comparison on the Mutillidae target. . . .	30
4.5	Phase 3 ablation test results on the Mutillidae target. Each row disables a single pipeline component while all others remain active.	34
4.6	Phase 4 lifecycle delta comparison on the controlled testbed, showing before (baseline) and after (remediated) scan results.	35
4.7	Longitudinal hallucination guard metrics across all evaluation scans where both <code>aiAnalysis</code> and <code>hallucinationGuard</code> were enabled.	37
4.8	Feature comparison across representative vulnerability assessment tools. Integration completeness is assessed rather than raw detection performance.	39
A.1	Supervisor meeting log with David Neilson.	51
B.1	Repository statistics at time of submission.	53

1. Introduction

The growth of AI-assisted software development has significantly increased development speed and productivity [30]. Large language models are now widely used to generate application code, automate documentation and even assist with debugging [30]. Due to this, research has strongly suggested that the increase of AI-generated code can not only introduce security vulnerabilities, insecure configurations but also poor validation practices [31].

As AI-assisted development increases, so does the potential attack surface of modern web systems. Vulnerabilities may be introduced faster than they are detected and remediated. Traditional security testing approaches, such as manual penetration testing and purple-team collaboration, aim to identify and reduce these risks [25]. However, these processes often involve manual triage, fragmented tooling and significant analyst effort.

Existing security tools can typically focus on isolated stages of the vulnerability lifecycle, such as detection or reporting [7]. Few systems show promise to integrate discovery, prioritisation, remediation guidance and reassessment within a single structured workflow [32]. AI can assist in explaining findings and generating remediation steps [15], [17], but unfortunately, AI systems introduce reliability risks such as hallucinated classifications or fabricated vulnerability references [10].

Consequently, there is a need for a structured framework that combines deterministic security analysis with controlled AI assistance, while maintaining validation mechanisms and supporting iterative reassessment. This project investigates whether a deterministic-first, AI-assisted purple-team framework could improve this lifecycle integration, maintain accurate vulnerability classification and provide reliable remediation guidance within authorised web environments.

1.1 Background & Context

Modern vulnerability management workflows operate across multiple disconnected stages, including automated discovery, severity scoring, contextual analysis, remediation planning and validation. Although each stage has matured independently, integration across the full lifecycle remains inconsistent. Security scanners can reliably extract technical artefacts such as open ports, service versions, CVE references, and misconfigurations. These tools are deterministic and reproducible, but their outputs are often verbose and lack contextual prioritisation or actionable remediation structure.

Severity scoring systems such as CVSS provide numerical estimates of impact and exploitability, yet they function as static metrics rather than adaptive prioritisation mechanisms. In operational environments, exploitability depends not only on severity but also on exposure context, service configuration and organisation risk tolerance. Consequently, analysts must interpret raw findings, determine relevance, and translate technical evidence into remediation tasks. This manual interpretation step introduces variability, increases cognitive load and reduces scalability.

Recent advances in large language models have introduced new possibilities for contextual interpretation of structured security data. Research suggests that LLMs can summarise vulnerability descriptions, explain technical findings in natural language, and propose remediation steps. However, these systems are inherently probabilistic. Unlike deterministic scanners, they may produce overconfident or even fabricated outputs if not constrained and validated. Reliability research has therefore emphasised structured prompting, schema enforcement and cross-verification mechanisms to mitigate hallucination risks.

Despite these parallel developments, the vulnerability lifecycle remains structurally fragmented. Deterministic scanners focus on extraction, scoring frameworks focus on ranking, AI systems focus on explanation, and validation research focuses on output verification. Few approaches integrate these elements into a unified workflow that preserves deterministic evidence while incorporating probabilistic assistance in a controlled manner. Additionally, reassessment after remediation is often treated as a separate operational task rather than an embedded lifecycle component.

This fragmentation creates a systemic gap between evidence generation and validated remediation. The absence of integrated lifecycle models limits the ability to combine reproducible classification, contextual prioritisation, structured AI guidance, hallucination validation and iterative reassessment within a coherent purple-team process.

Addressing this gap requires a framework that treats vulnerability management as a continuous pipeline rather than a sequence of isolated tools. Such an approach must preserve deterministic reliability while embedding controlled AI interpretation and explicit validation layers to ensure trustworthiness across the full lifecycle.

1.2 Research Questions

Based on the identified research gap, this study is guided by the following research questions:

- I. To what extent can a deterministic-first, AI-assisted purple-team framework inte-

grate vulnerability discovery, prioritisation, and structured remediation guidance within a single reproducible pipeline?

- II. How consistently can a hybrid deterministic and LLM-assisted classification mechanism map identified vulnerabilities to OWASP Top 10 categories across heterogeneous authorised test targets?
- III. Can structured hallucination mitigation through deterministic cross-validation measurably constrain AI-generated classification inconsistencies and fabricated vulnerability references within an automated vulnerability assessment pipeline?

1.3 Aims

The aim of this research is to design, implement, and empirically evaluate a deterministic-first, AI-assisted purple-team framework that integrates vulnerability discovery, exploitability-aware prioritisation, structured remediation guidance, hallucination mitigation, and iterative reassessment within a unified lifecycle model.

1.3.1 Objectives

- A. To design a modular, reproducible vulnerability scanning pipeline.
- B. To implement deterministic OWASP Top 10 classification logic.
- C. To integrate a structured LLM-assisted analysis component with enforced JSON schema validation and output constraints.
- D. To implement a hallucination detection and trust scoring mechanism that cross-validates AI output against deterministic scan evidence.
- E. To evaluate classification consistency, remediation quality, and OWASP coverage across a heterogeneous set of authorised test targets.
- F. To implement structured scan delta comparison supporting iterative reassessment.
- G. To introduce exploitability-aware contextual weighting into the security scoring model.

1.3.2 Deliverables

The primary deliverable of this research is a fully functional Electron-based desktop application implementing the proposed deterministic-first, AI-assisted purple-team framework.

The system consists of a modular pipeline architecture incorporating a structured vulnerability scanning engine, deterministic OWASP Top 10 classification module, exploitability-aware security scoring algorithm, AI-assisted remediation guidance component with enforced schema validation and a structured hallucination validation and trust scoring layer. The framework further includes an exportable PDF reporting system, iterative scan comparison and change summary module, with a hallucination evaluation metric logging component to support empirical analysis.

Supporting deliverables include an evaluation dataset derived from authorised vulnerable environments and a defined empirical evaluation framework for measuring classification accuracy, remediation reliability, OWASP coverage, and validation performance.

2. Literature Review

2.1 Introduction

Designing an automated vulnerability assessment pipeline requires making a series of constrained engineering decisions, and those decisions can only be made responsibly with an honest account of what prior work has achieved and where it falls short. This chapter surveys the existing literature across six themes that correspond directly to the pipeline stages of concern in this project: the maturity and limitations of automated scanning, the reliability of vulnerability scoring systems, the operational role of AI-assisted classification, the state of automated remediation, the tractability of LLM hallucination control, and the degree to which the purple-team lifecycle has been integrated in practice.

The purpose is not to summarise the field in isolation but to trace a line of cumulative constraint, from what detection can reliably produce, through what classification and scoring can responsibly claim, to what remediation and validation can currently deliver, because understanding that line of constraint is what determines what a new system needs to do differently and why.

2.2 Detection Maturity and the Limits of Automated Scanning

Any pipeline that begins with automated scanning inherits the structural characteristics of the tools it depends upon, and those characteristics are worth understanding precisely rather than assuming. Singh et al. validate Nmap as a scalable discovery instrument capable of high detection accuracy across simulated environments of varying complexity, which provides empirical grounding for treating it as a credible backbone for automated pipelines [31]. This confirmation matters because the choice of scanner determines the shape of every downstream stage: the parser must accommodate its output format, the OWASP mapping layer must account for its coverage boundaries, and the scoring model must be calibrated against what it can and cannot find. Chalvatzis et al. extend the comparative picture by showing that Nmap completes network assessments faster than Nessus and OpenVAS, though at the cost of reduced CVE-linked coverage [7]. This means that speed and coverage sit in a genuine tension rather than being simultaneously optimisable, a trade-off that a two-phase progressive scan design can partially manage but not eliminate.

The limitations become more constraining when the scope moves from network-layer detection to web application logic. Althunayyan et al. quantify these constraints directly, finding that OWASP ZAP achieved 100% precision against injection vulnerability classes but only 14 to 29% recall, where recall measures the proportion of actual vulnerabilities the scanner successfully detected, with Burp Suite performing at a similar level of recall failure [3]. The cause is a structural one: black-box scanners cannot maintain authenticated session state or traverse modern JavaScript-heavy application flows. Systematic false negatives for authenticated endpoints and sophisticated injection types are therefore not a correctable deficiency but an inherent boundary condition that any downstream process must accommodate.

Mateo-Tudela et al. demonstrate that combining static, dynamic, and interactive testing paradigms improves the situation, with recall reaching approximately 70% for XSS and 60% for SQL injection, but note that even combined approaches leave a substantial proportion of attack surface undetected and that the correlation of results across tool types is not yet well supported by automated evaluation tooling [24]. Abdulghaffar et al. address the orchestration question directly, finding that running multiple scanners within a single platform improves aggregate coverage but introduces noise and inter-tool contradiction that require dedicated consolidation logic rather than simple result merging [1]. Redondo and Cuesta reinforce the operational conclusion, demonstrating that structured task representation and deterministic parsing of scanner output are conditions for repeatable, correct assessments rather than incidental implementation choices [28].

Scanner outputs are incomplete by design, noisy in practice and require disciplined post-processing before they can serve as the evidence base for classification or scoring. None of the reviewed works integrates downstream classification, scoring, or remediation into the same evaluation, leaving the field having characterised what a pipeline can ingest but not what it should do with that input.

2.3 Risk Scoring Models

Translating a set of raw vulnerability findings into an ordered remediation priority requires a scoring layer, and the decision of how to design that layer is directly shaped by the documented failures of existing scoring systems. Howland argues that CVSS, despite its near-universal adoption, conflates additive severity with topological exploitability risk, producing prioritisation decisions that are systematically miscalibrated against actual exposure [13]. The practical consequence is that organisations applying CVSS scores directly to remediation queues will consistently over-prioritise low-exposure findings and under-prioritise findings where exploitability is high but the base score is

moderate.

Jacobs et al. support this empirically, showing that static severity scores are weak predictors of actual exploitation in the wild and that incorporating data-driven exploit probability signals substantially improves prioritisation accuracy [14]. The implication for any scoring model is that severity alone is an insufficient signal and that contextual exposure factors, including port accessibility, service type, and the presence of authentication weaknesses, need to be weighted alongside raw score values.

Koscinski et al. extend the critique by demonstrating that different scoring systems produce conflicting signals on the same dataset, concluding that any scoring mechanism should be treated as a transparent decision-support tool operating within acknowledged limitations rather than as an objective measure of risk [18]. Lei et al. propose dependency-aware probabilistic approaches to address aggregation across attack paths [21], though attack graph modelling of this kind requires network topology data unavailable in black-box web scanning contexts, and a scoring model for this pipeline must therefore remain operable without source access.

Wen and Katt demonstrate that grounding scoring in the OWASP Application Security Verification Standard can provide checkable acceptance criteria and a structured quantitative evaluation framework [33]. Sane provides an important limiting condition, showing empirically that the OWASP Top 10 is not a superset of commonly reported weaknesses in the National Vulnerability Database. Any taxonomy dependent on it must therefore be designed with the awareness that its coverage is partial rather than exhaustive [29].

The scoring literature, read together, points towards a scoring model that incorporates weighted exploitability proxies, treats OWASP mapping as a useful but bounded classification layer, and makes its assumptions explicit rather than presenting its outputs as objective risk scores, which is both a design requirement and a limitation that needs to be acknowledged during evaluation.

2.4 AI-Assisted Classification

The shift from rule-based vulnerability classification towards machine learning, and subsequently towards large language models, represents a genuine capability expansion, though the conditions under which that expansion is operationally useful are considerably more constrained than early work suggested. Le, Chen, and Babar provide the taxonomy that situates this shift, tracing the movement from static rule sets through classical ML-based prioritisation [20]. Their work establishes the research landscape within which LLM-based approaches now operate.

Pan et al. demonstrate that automated CWE classification using ML classifiers is feasible and outpaces manual labelling in lexical coverage, but note that classifiers drift, learn spurious correlations from training data, and are difficult to audit post-deployment [26]. The auditability problem is relevant to operational use because a classifier that cannot be inspected or constrained introduces an uncertainty that compounds with the uncertainty already present in scanner output, and the combined uncertainty is what the pipeline must manage.

Lin et al. conduct a comparative evaluation of large language models on software vulnerability detection tasks, finding that whilst LLMs can perform classification, performance varies significantly across models, and the models frequently fail to produce explanations that are operationally actionable [22]. This finding motivates a specific architectural decision, namely that scan discovery evidence needs to be kept structurally separate from LLM summarisation, because allowing the model to reason freely over unstructured scanner output risks producing inferences that are not grounded in what the scanner actually found.

Chen et al. provide empirical support for this concern in the adjacent domain of smart contract detection, finding that ChatGPT achieves high recall but limited precision, with false positive costs that carry direct operational consequences [8]. Neither study provides a mechanism for constraining model errors at inference time. The validation question therefore remains open, and it determines whether LLM integration into a security pipeline reduces or increases the triage burden on the analyst.

The operational cost of unvalidated LLM output is characterised in detail by Alahmadi et al., whose qualitative SOC study finds that alert noise materially degrades analyst performance, increases cognitive fatigue, and causes genuine threats to be missed [2]. Kersten et al. extend this, demonstrating that investigation volume so constrains analyst workflow that concise, grounded signal extraction is an operational requirement rather than a design preference [16]. Taken together, these findings establish that AI-assisted classification is only operationally beneficial when the model's output is constrained to what the underlying evidence can support, which is a validation architecture requirement rather than merely a quality improvement.

2.5 Remediation Automation

Automated remediation is the stage at which the gap between research capability and deployed reality is most pronounced, and understanding the shape of that gap is necessary before claiming to address it. Bhandari et al. provide one of the more rigorous treatments, tying LLM-generated patches to observable developer commit behaviour and

demonstrating that specialised patch models outperform generic LLMs when correctness is evaluated against real acceptance criteria [4]. Xu et al. complement this by showing that single-pass remediation generation consistently produces low-quality output and that iterative refinement loops with structured objectives produce measurably better results [34].

Both contributions are significant, and both carry a constraint that limits their applicability to the present context: they assume access to the application’s source code, which is a condition that does not hold in black-box web scanning. When the pipeline has no visibility into the codebase, remediation must be expressed as natural-language configuration guidance covering headers, TLS settings, authentication policies, and exposure reduction, rather than as generated patches. The correctness criteria, the evaluation methodology, and the validation requirements are therefore all different from those the patch generation literature addresses.

The SOAR literature approaches this from the workflow side rather than the code generation side. Bridges et al. establish a precedent for evaluating security automation against human-centred metrics rather than purely technical ones, supporting the position that automated remediation quality should be assessed in terms of its effect on workflow efficiency and analyst burden rather than correctness alone [6]. Alahmadi et al. reinforce this from the analyst perspective, establishing that precision and structured parsing are operational safety requirements whose absence has a measurable cost in missed detections and increased fatigue [2]. The implication is that a remediation component generating plausible-sounding guidance that is ungrounded in scan evidence does not simply fail to help but actively degrades the signal quality that the rest of the pipeline has worked to establish.

What the remediation literature as a whole reveals is a structural fragmentation: patch generation work does not address hallucination control, SOAR work does not address LLM-generated natural-language guidance, and no reviewed work closes the full loop from scanner output through classification, scoring, and validated natural-language remediation within a single deployable system. That fragmentation is what defines the implementation challenge.

2.6 Hallucination and LLM Reliability

The hallucination problem in large language models is not a peripheral concern for a pipeline that uses LLM output as a component of a security assessment but a fundamental design constraint, because a remediation suggestion that is confidently stated but factually incorrect is not a neutral output. Rawte et al. characterise

hallucination as a multi-modal failure requiring detection and constraint at the system level, arguing that guardrail architectures must reason about the model’s underlying uncertainty rather than checking only the surface plausibility of its final outputs [27]. The distinction matters in practice because a model can produce a syntactically coherent, contextually appropriate remediation recommendation that is nevertheless unsupported by anything the scanner found, and surface-level plausibility checks will not catch that failure.

Farquhar et al. operationalise a detection approach, demonstrating that semantic entropy, understood as a measure of uncertainty over meaning rather than over tokens, can serve as a principled signal for output restriction or abstention, offering a mechanism to suppress remediation claims that the model cannot reliably ground [10]. The translational challenge they acknowledge is that converting semantic entropy into scanner-evidence checks requires domain-specific design work that their general-purpose framework does not provide. Luo et al. introduce a different framing, demonstrating that hallucination behaviours can be actively exploited to produce vulnerable outcomes in generated code, which reframes hallucination not as an accuracy problem but as a security risk in contexts where the pipeline’s output is acted upon directly [23]. Dong et al. synthesise the guardrail vocabulary across the literature, formalising grounding, validation, and abstention as the three functional components of a reliable LLM architecture and concluding that effective guardrails must be layered and updated continuously [9]. Hammar et al. bring this closest to a deployable design, treating hallucination control as a systems engineering problem with tunable consistency thresholds and demonstrating that restricting LLM output to claims consistent with an external evidence channel reduces both hallucination incidence and operational recovery time [11].

The consistent finding across this section of the literature is that LLM integration into operational pipelines is viable under the right architectural conditions, and that the key condition is the existence of a grounded external evidence channel against which model outputs can be validated. The reviewed works do not, however, use the structured output of a live vulnerability scanner as the evidence channel against which LLM claims are validated, which is precisely the approach taken in the hallucination guard described in Chapter 3.

2.7 Purple Team Lifecycle Integration Gaps

The purple-team lifecycle, understood as the coordinated and iterative cycle in which offensive and defensive activities inform each other rather than operating in isolation, provides the operational context within which an automated assessment pipeline ultimately needs to function, and the literature in this area is informative both for what it

has addressed and for what it has not.

Holm and Sommestad demonstrate empirically that automated threat emulators can preserve the learning value of red team exercises without the resource overhead of human attackers, with analysts unable to reliably distinguish automated activity from human-directed action in controlled exercises [12]. Landauer et al. extend this by showing that automated attack execution can be designed to produce log artefacts statistically closer to those of human attackers, which is relevant because automation that generates easily attributable, unrealistic traces undermines the defensive feedback value of the exercise [19]. Bhattacharya et al. model offensive action selection as a reinforcement learning optimisation problem, producing attack sequences that outperform greedy baselines, a framing that is conceptually applicable to the sequencing of scanning phases as well as to adversary emulation [5].

None of the reviewed works integrates automated scanning, deterministic classification, AI-assisted summarisation, hallucination validation, and comparative reporting within a single framework that closes the loop from initial reconnaissance through to exportable findings, and this structural gap is what this project is designed to address.

2.8 Summary and Transition to Methodology

The six themes reviewed in this chapter converge on a consistent finding: each stage of the vulnerability assessment pipeline has been studied in some depth, but the pipeline as an integrated whole has not. Automated scanners produce incomplete, noisy output that requires structured post-processing before it can serve as an evidence base [1], [3], [28], [31]. Existing scoring systems conflate severity with exploitability in ways that lead to miscalibrated prioritisation, and the literature supports weighted, context-aware alternatives that treat their own limitations transparently [13], [14], [18]. AI-assisted classification expands detection coverage but introduces false positive costs that are operationally damaging unless constrained by a validation architecture that keeps model inferences grounded in scanner evidence [2], [8], [22]. Automated remediation in black-box web scanning contexts requires natural-language configuration guidance rather than code generation, and no reviewed work addresses this combination within a validated pipeline [4], [34]. The hallucination problem in LLMs is tractable under the right architectural conditions, but the reviewed works do not ground validation in the specific evidence channel that a scanner-based pipeline provides [10], [11], [23], [27]. The purple-team lifecycle, whilst increasingly well supported on its offensive side, lacks an integrated framework that closes the loop through to validated, exportable defensive findings [11], [12], [19].

These gaps, taken together, define the implementation requirements for the system described in the following chapter.

3. Methodology

3.1 Research Approach

This project follows a development-oriented research methodology with quantitative empirical evaluation. The primary output is a working application, and the evaluation criteria are drawn from measurable performance indicators rather than subjective interpretation. This approach is appropriate because the research questions posed in Chapter 1 ask whether a deterministic-first, AI-assisted framework can maintain classification accuracy, provide structured remediation guidance and support iterative reassessment, and answering those questions requires building the system and then measuring what it produces under controlled conditions.

The research employs primary data collection through controlled experimentation. All evaluation data is generated firsthand by executing the framework’s pipeline against authorised vulnerable targets and recording the outputs at each stage. The data is quantitative: numerical scores, classification rates, hallucination metrics and vulnerability counts provide the evidence base for every evaluative claim made in Chapter 4. This positions the work within the experimental computing tradition identified in the module’s research methodology guidance, where a prototype is developed, tested against defined criteria and evaluated against the constraints the literature establishes.

The alternative approaches considered and rejected were survey-based evaluation, which would introduce subjective analyst opinion rather than reproducible measurement, and purely theoretical analysis, which could not demonstrate whether the proposed integration actually works in practice. The development-and-test approach was selected because the research gap identified in Chapter 2 is fundamentally an integration gap, and proving that an integration can work requires demonstrating it empirically rather than arguing for it in the abstract.

3.2 System Design

3.2.1 Requirements

The system was designed against the seven objectives defined in Section 1.3.1. Table 3.1 maps each objective to its corresponding pipeline module, classifies the requirement type and identifies the evaluation phase in which its fulfilment is verified. This traceability ensures that every component of the implementation has a defined purpose grounded in the research aims, and that every evaluative claim in Chapter 4 can be traced back to a stated requirement.

Table 3.1: Requirements traceability matrix mapping proposal objectives to system modules and evaluation phases.

Obj.	Requirement	Module	Type	Evaluation
A	Modular, reproducible scanning pipeline	Scanner (<code>nmap.ts</code> , <code>parser.ts</code>)	Func.	Phase 1: consistency (B1 vs B2)
B	Deterministic OWASP Top 10 classification	Analysis (<code>owasp-mapper.ts</code>)	Func.	Phase 3: ablation (A1)
C	Structured LLM-assisted analysis component with JSON schema enforcement	LLM (<code>gemini.ts</code>)	Func.	Phase 3: ablation (A2)
D	Hallucination detection and trust scoring, cross-validating AI output against deterministic scan evidence	Analysis (<code>hallucination-guard.ts</code>)	Non-functional	Phase 5: longitudinal (H1–H4)
E	Evaluate classification consistency, remediation quality and OWASP coverage across a heterogeneous set of authorised test targets	Evaluation framework	Proc.	Phases 1–5
F	Structured scan delta comparison	Analysis (<code>delta-comparison.ts</code>)	Func.	Phase 4: lifecycle (L1–L4)
G	Exploitability-aware scoring	Analysis (<code>security-scorer.ts</code>)	Func.	Phase 2: comparative (E1, E2)

Three requirement types are distinguished. Functional requirements (A, B, C, F, G) specify capabilities the system must implement and are verified by demonstrating correct output under controlled conditions. The non-functional requirement (D) specifies a quality attribute, namely that AI-generated outputs must be validated to a measurable trust threshold, and is verified through longitudinal metric accumulation. The procedural requirement (E) defines the evaluation methodology itself and is fulfilled by the five-phase framework described in Section 3.3.

3.2.2 Pipeline Architecture

PurpleTeam Suite implements a ten-stage linear pipeline in which the output of each stage becomes the input of the next. The system is built as an Electron desktop application using React and TypeScript, with all backend logic isolated from the user interface through Electron’s IPC bridge. This separation is a deliberate security constraint: the renderer process has no direct access to the file system, network scanning tools or external APIs, and all communication passes through pre-approved channels registered in the main process.

Figure 3.1 illustrates the full pipeline architecture. Each stage is implemented as an independent module with its own type definitions and barrel export, meaning any module can be replaced, extended or disabled without affecting adjacent stages. The dashed feedback path in the diagram represents the iterative scan-remediate-rescan lifecycle supported by the delta comparison system described in Section 3.2.6.

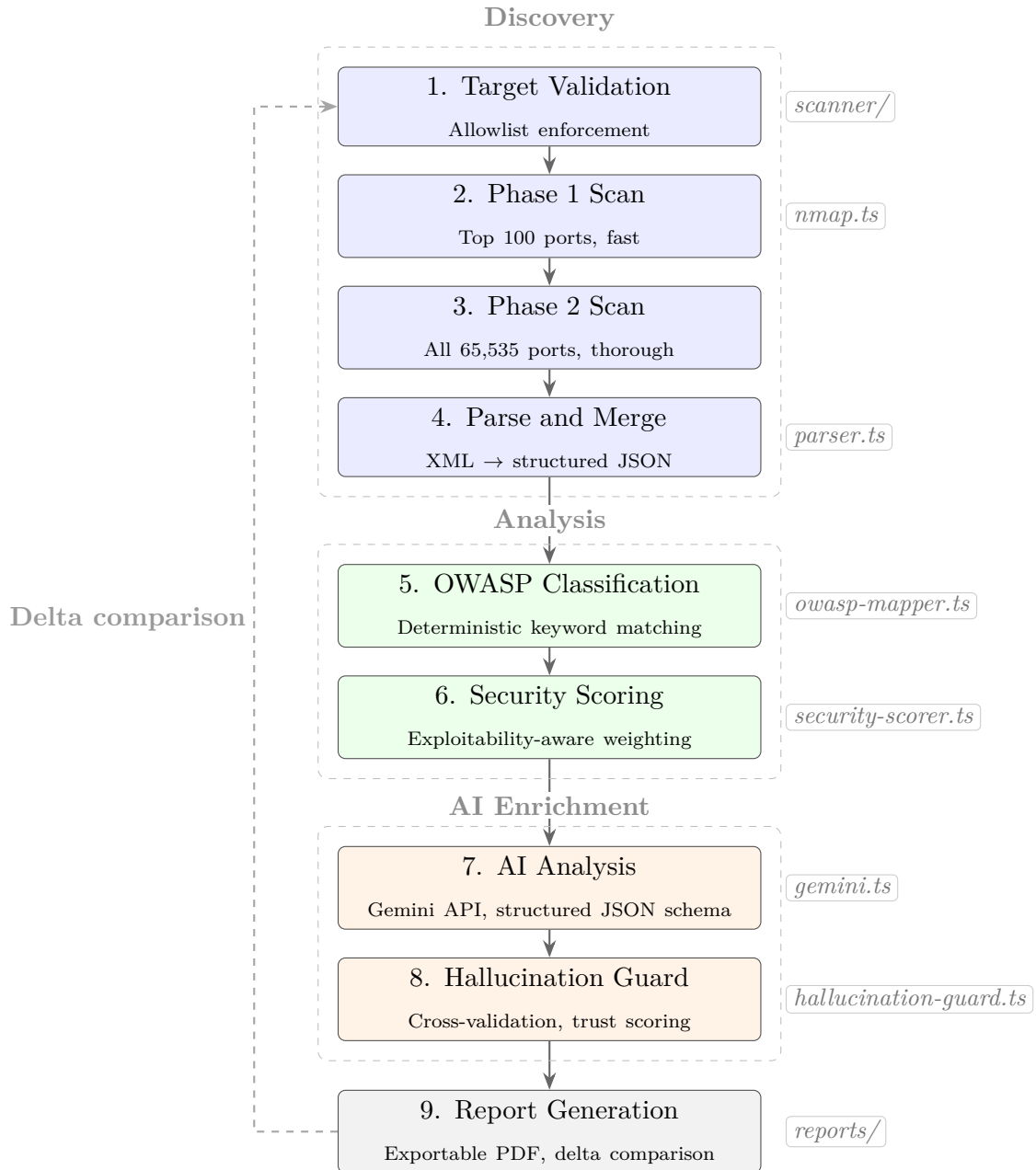


Figure 3.1: PurpleTeam Suite pipeline architecture. Each stage is implemented as an independent module. The dashed feedback arrow represents the iterative scan-remediate-rescan lifecycle supported by the delta comparison system.

3.2.3 Scanning and Detection

The scanner module orchestrates Nmap as an external child process, using a two-phase progressive design to balance responsiveness against coverage [7]: Phase 1 scans the top 100 ports and returns early results within approximately two to four minutes, while Phase 2 runs a full 65,535-port scan in the background, with results merged and deduplicated after both phases complete. The parser additionally injects post-processing detections for missing security headers and absent HTTPS, extending coverage into

configuration-level findings that Nmap’s scripting engine does not natively report.

The screenshot displays a web interface for a 'Target Scanner'. At the top, there are three tabs: 'SCAN' (active), 'RESULTS', and 'REPORTS'. Below the tabs, the 'Target Scanner' section prompts the user to 'Enter a target URL or IP address to initiate vulnerability scanning.' A text input field contains 'localhost:8080', and a 'VALIDATE' button is to its right. Below this, a message states 'Enter any target URL or IP address to scan' with a '✓ Known safe target' indicator. The 'Confirm Scan' section informs the user: 'You are about to scan localhost:8080. This will run a progressive scan: quick discovery first, then deep vulnerability testing.' It features 'START SCAN' and 'CANCEL' buttons. At the bottom, the 'SCAN PROGRESS' section shows a six-step stepper: 1. VALIDATING (checked, 'Checking target'), 2. CONFIRM (selected, 'Awaiting user action'), 3. PHASE 1 (radio button, 'Quick discovery'), 4. PHASE 2 (radio button, 'Deep vulnerability scan'), 5. ANALYZING (radio button, 'AI processing'), and 6. COMPLETE (radio button, 'Results ready').

Figure 3.2: Scan target input interface with allowlist validation. The six-step progress stepper is visible prior to scan execution.

3.2.4 Classification and Scoring

The classification layer uses a deterministic keyword matcher to map each vulnerability to OWASP Top 10 categories; it runs before the AI component, establishing a ground-truth classification that the hallucination guard later cross-references. The scoring model addresses the limitations of static CVSS-based prioritisation [13], [14] by applying four contextual multipliers: port exposure weighting, service exposure, an authentication weakness multiplier, and a flat TLS absence penalty. All multipliers are defined as named constants and are fully deterministic; a runtime feature toggle allows contextual weighting to be disabled for ablation testing. A flat +10 remediation bonus is applied when AI analysis completes successfully, reflecting full pipeline execution. Figure 3.3 presents the dashboard view of a completed scan.

3.2.5 AI Integration and Hallucination Control

The LLM component sends compressed vulnerability data to Google Gemini under structured JSON output enforcement and a constrained response schema, ensuring the model analyses only what the scanner found [8], [22]. The hallucination guard cross-references AI OWASP assignments against the deterministic mapper’s classifications, verifies that cited CVE identifiers appear in the scan data, and computes a weighted trust score that penalises fabricated CVEs most heavily, OWASP disagreements moderately,

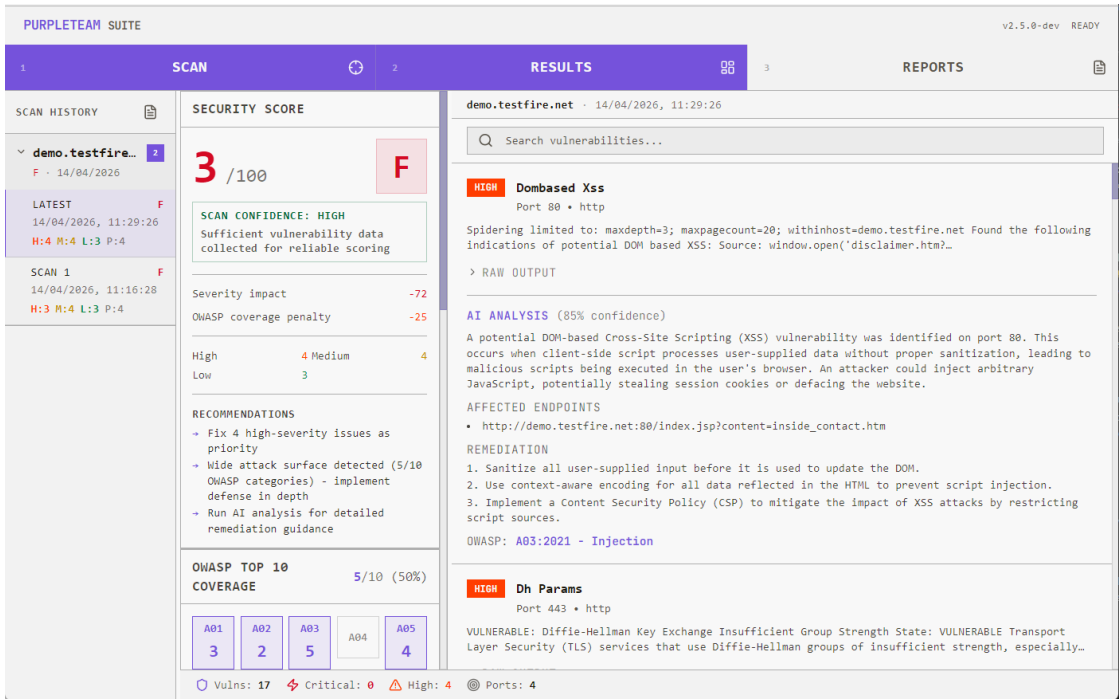


Figure 3.3: PurpleTeam Suite dashboard displaying security score with exploitability-aware weighting, OWASP Top 10 coverage matrix and prioritised vulnerability table.

and confidence mismatches least. Trust scores are clamped to the 0–100 range and persisted longitudinally to enable reliability tracking across scans. Figure 3.4 illustrates the AI-generated analysis for a single vulnerability.

3.2.6 Lifecycle Support

The delta comparison module closes the purple-team lifecycle loop identified as a gap in the literature [12] by fingerprinting vulnerabilities across consecutive scans and classifying each finding as resolved, persisting or newly introduced, with score changes, OWASP coverage shifts and directional indicators presented in both the dashboard and an exportable report. Figure 3.5 shows the in-app comparison view and the exported report used to evidence lifecycle impact in Section 4.5.

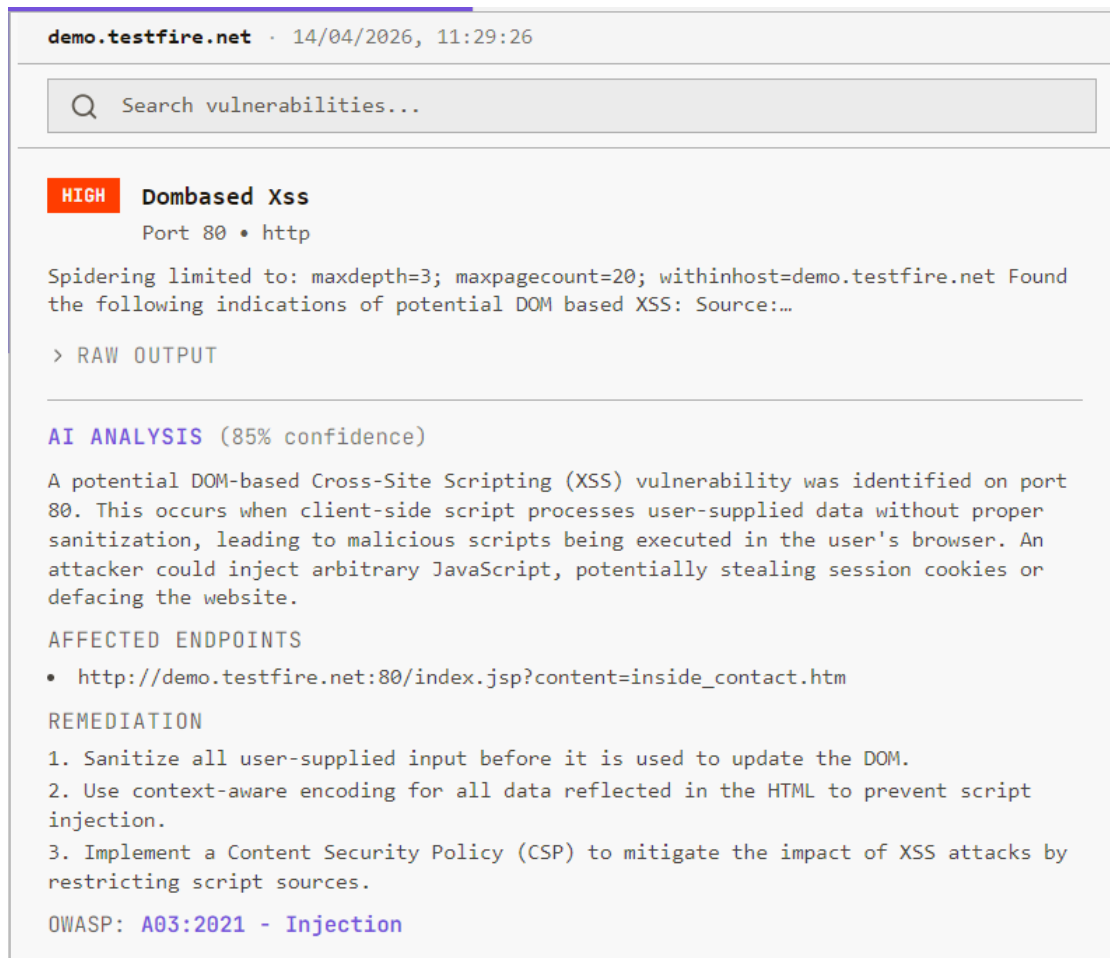
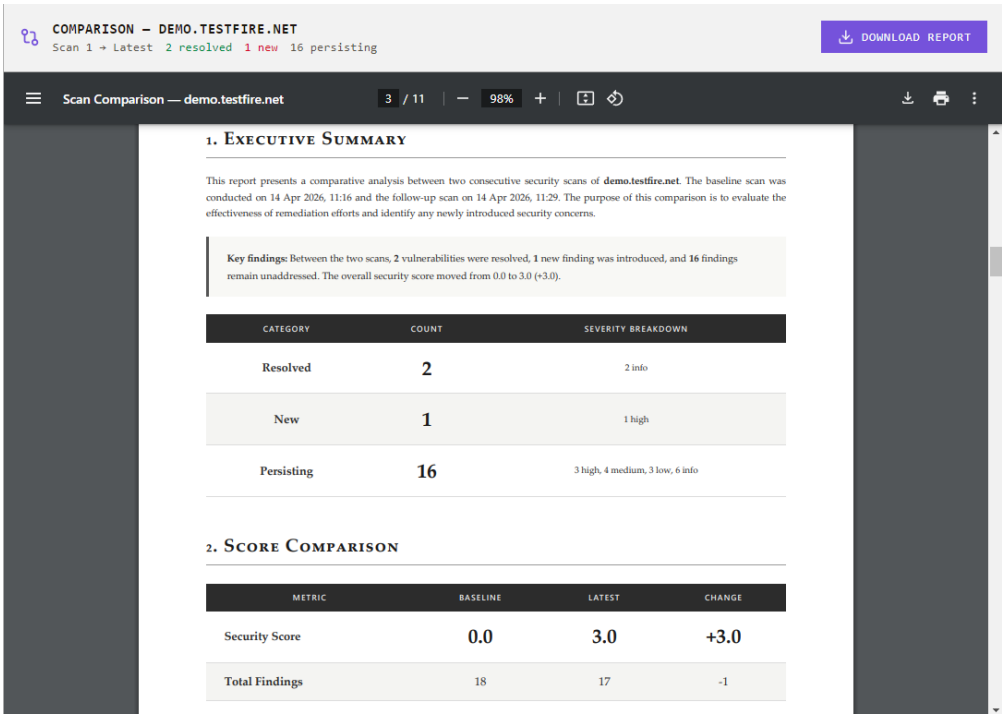
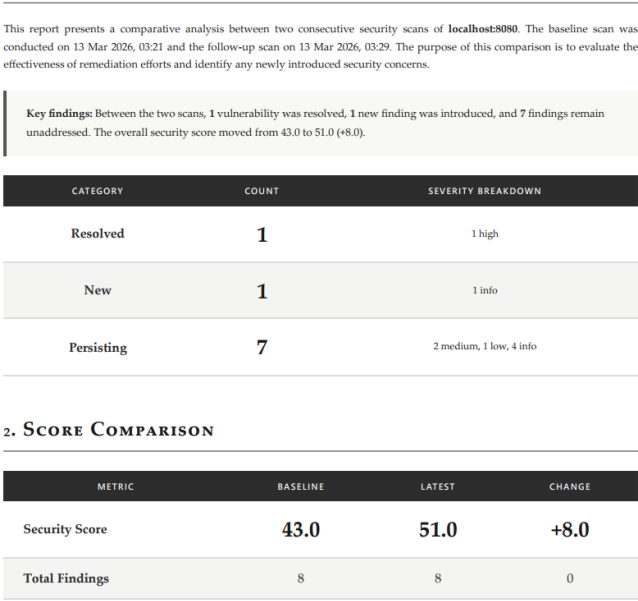


Figure 3.4: AI-generated vulnerability analysis showing plain-English summary, severity justification and structured remediation steps. The hallucination guard trust indicators are displayed alongside each finding.



(a) In-app comparison report executive summary view (toggle-driven delta example).



(b) Exported localhost comparison report excerpt showing a +8 score improvement after local target changes; this screenshot predates the definitive lifecycle evaluation reported in Section 4.5.

Figure 3.5: Delta comparison outputs in PurpleTeam Suite, showing both the application report interface and the exported report evidence used in lifecycle evaluation.

3.2.7 Implementation Environment

Table 3.2 and Table 3.3 summarise the technology stack and key dependencies; selection rationale and architectural justification are provided in Appendix C.

Table 3.2: Core technology stack and selection rationale.

Layer	Technology	Justification
Desktop framework	Electron	Compiles to native executables for Windows, macOS and Linux from a single TypeScript codebase; provides secure IPC isolation between renderer and main processes, ensuring the UI cannot directly access the file system or spawn scanning tools
Frontend	React 19, TypeScript	Component-based UI architecture with static type checking, reducing runtime errors in security-critical data handling
Styling	Tailwind CSS	Utility-first approach enabling rapid, consistent UI development without external stylesheet dependencies
Build tooling	Vite	Sub-second hot module replacement during development; native TypeScript and JSX support without additional transpilation configuration
Scanning	Nmap (external)	Industry-standard network scanner validated for automated pipeline use [31]; spawned as a child process to maintain separation from the application runtime
XML parsing	xml2js	Converts Nmap's XML output into traversable JavaScript objects; mature library with established handling of Nmap's output schema
AI provider	Google Gemini 2.5 Flash	Structured JSON output mode, low-latency inference, configurable temperature, and sufficient context window for batch vulnerability analysis
Routing	React Router	Client-side navigation between scan, dashboard and reports views without full page reloads
State management	Custom singleton store	Lightweight in-memory store avoiding external state library overhead; persists across page navigation through React hooks subscription
PDF generation	jsPDF (programmatic)	Enables server-side PDF generation within the Electron main process without browser print dependencies

Table 3.3: Key library dependencies and pipeline integration points.

Library	Pipeline Stage	Role
xml2js	Parse (Stage 3)	Parses Nmap XML into typed JSON; handles nested script output and multi-valued fields
@google/generat	AI Analysis (Stage 8)	Official Gemini SDK; provides structured output mode, temperature control, token limits and retry configuration
jspdf	Report (Stage 10)	Generates multi-page PDF reports programmatically with custom layouts, tables and embedded score visualisations
electron	Application shell	Main and renderer process management, IPC bridge, context isolation, system dialog access for file export
react-router-dc	UI navigation	Declarative routing between scan, dashboard and reports pages
dotenv	Configuratio	Loads Gemini API keys from <code>.env</code> file, keeping credentials outside the codebase and version control

3.3 Evaluation Framework

The evaluation is structured across five phases, each producing quantitative data against defined metrics across five authorised test targets representing a range of technology stacks, deployment models and vulnerability profiles. Phase 1 establishes baseline consistency; Phase 2 compares flat versus contextual weighting; Phase 3 performs per-component ablation on Mutillidae using a runtime feature toggle system; Phase 4 demonstrates the iterative lifecycle on the controlled testbed; and Phase 5 accumulates longitudinal hallucination guard metrics across all targets. The five phases are summarised in Table 3.4.

#	Phase	Targets & Config.	Method	Key Output Metrics
1	Baseline Consistency	All five targets Weighting: OFF 2 runs per target	Run the full pipeline twice per target. Verify identical inputs produce identical outputs.	Baseline scores B_1, B_2 · Scan consistency rate · OWASP coverage breadth
2	Exploitability-Aware Scoring Impact	All five targets Weighting: ON 1 run per target	Re-run with contextual weighting enabled. Compare adjusted scores against Phase 1 baselines.	Score delta ΔE vs Phase 1 · Per-multiplier breakdown · Adjusted vs flat score table
3	Ablation Testing	localhost only Weighting: toggle No re-scan required	Disable one component per run and reprocess the same scan data. Components: OWASP mapper, AI analysis, hallucination guard, contextual weighting.	Per-component score and classification delta · Ablation table across four toggle conditions
4	Lifecycle Demonstration	localhost only Weighting: ON Scan → fix → rescan	Remediate a defined vulnerability subset, rescan, and generate a delta comparison report.	Score improvement ΔS · Resolved and persisting counts · OWASP coverage contraction
5	Longitudinal Hallucination Reliability	All five targets Weighting: ON Accumulated across scans	Accumulate hallucination guard metrics across all targets and scans to produce a longitudinal reliability dataset.	Trust score · OWASP disagreement rate · Confidence mismatch rate · Fabricated CVE count

Table 3.4: Five-phase evaluation framework with target configurations, methods and key output metrics.

3.4 Limitations and Ethical Considerations

The system carries several acknowledged limitations. The target set is small, comprising five authorised environments, which limits the generalisability of evaluation findings. The AI component depends on a single LLM provider, and model behaviour may differ across versions or alternative providers. The contextual weighting multipliers were calibrated based on common exposure patterns rather than empirical attack frequency data, and should be treated as operationally reasonable defaults rather than statistically validated risk weights.

The system performs active network scanning, which is regulated under the Computer Misuse Act 1990. Accordingly, all scanning was restricted to authorised targets through an enforced allowlist validation mechanism that prevents scan execution against unapproved hosts. All testing was restricted to the five authorised targets described in Section 3.3. No live production systems were tested and no human participants or personal data were involved. Ethical approval was granted by the project supervisor on 11 February 2026.

Only compressed vulnerability metadata is transmitted to the Gemini API. Raw scan outputs, network topology data and sensitive system information are not uploaded, limiting external data exposure to the minimum necessary for AI analysis. The hallucination guard further ensures that AI-generated claims are not presented to the user without deterministic cross-validation, reducing the risk of acting on fabricated security advice. The application displays a legal disclaimer regarding the implications of misuse, and responsible operation requires explicit authorisation from the target owner.

3.5 Summary

This chapter established the methodological foundation for the evaluation that follows. The system is designed against seven traceable requirements, each mapped to a pipeline module and evaluation phase, with a runtime feature toggle system enabling controlled ablation without modifying the codebase. The five-phase evaluation framework covers baseline consistency, exploitability-aware scoring impact, per-feature contribution isolation, iterative lifecycle demonstration and longitudinal hallucination reliability. The system is feature-complete, and results are presented in Chapter 4.

4. Analysis and Results

4.1 Evaluation Design Overview

The evaluation was conducted according to the five-phase framework defined in Section 3.3. Table 4.1 summarises the five authorised targets; Phases 1, 2 and 5 ran across all five, Phase 3 (ablation) was conducted on Mutillidae only, and Phase 4 (lifecycle delta) on the controlled testbed only, the sole target with researcher-controlled vulnerability state.

Table 4.1: Evaluation targets and their roles within the five-phase framework.

Label	Address	Type and Role
Mutillidae	localhost (port 80)	Local, OWASP Docker. Mutillidae II running at Security Level 0, exposing Apache, PHPMyAdmin, OpenLDAP and HTTPS across ports 80, 81, 82, 389 and 443. Used for Phases 1, 2, 3 and 5.
Controlled Testbed	127.0.0.1:8080	Local, custom Node.js. Purpose-built application with 12 intentional vulnerabilities across multiple OWASP categories. Sole target with researcher-controlled vulnerability state. Used for Phases 1, 3 and 4.
scanme.nmap.org	scanme.nmap.org	Public, Nmap Project. Explicitly authorised for port scanning, minimal service exposure provides a clean external reference. Used for Phases 1 and 5.
testasp.vulnweb.com	testasp.vulnweb.com	Public, Acunetix test site (IIS/ASP/MSSQL). Technology profile distinct from the local PHP targets. Used for Phases 1 and 5.
demo.testfire.net	demo.testfire.net	Public, HCL/IBM banking app (IIS/ASP.NET). Deliberately vulnerable financial-services attack surface. Used for Phases 1 and 5.

All scans were conducted on a Windows 11 workstation with Nmap 7.98 on the system PATH; the `featureToggles` snapshot persisted with each result ensures full configuration traceability.

4.2 Phase 1: Baseline Consistency and Reproducibility

Severity deduction scores range from 50 to 181 across the five targets, with four receiving an F grade and only testasp.vulnweb.com achieving a D; Mutillidae’s 26 findings across all severity tiers justify its selection as the primary ablation target in subsequent phases (Table 4.2, Figure 4.1). Four of the five targets achieved 4/10 OWASP category coverage with detections concentrated in A01, A02, A03 and A05, with demo.testfire.net reaching 5/10 by additionally triggering A07 detections consistent with its financial-application authentication endpoints (Table 4.3, Figure 4.3).

Table 4.2: Phase 1 baseline scan results across all five evaluation targets (all feature toggles enabled).

Target	Sev. Ded.	Grade	Crit.	High	Med.	Low	Info	Total
Mutillidae	181	F	2	5	4	3	12	26
Controlled Testbed	77	F	2	3	2	0	8	15
scanme.nmap.org	91	F	2	1	1	1	1	6
demo.testfire.net	96	F	3	4	3	0	8	18
testasp.vulnweb.com	50	D	2	1	1	1	0	5

Table 4.3: OWASP Top 10 category distribution across baseline scans.

Target	A01	A02	A03	A04	A05	A06	A07	A08	A09	A10	Cov.
Mutillidae	2	5	7	0	12	0	0	0	0	0	4/10
Controlled Testbed	2	5	0	0	6	0	2	0	0	0	4/10
scanme.nmap.org	1	3	1	0	1	0	0	0	0	0	4/10
demo.testfire.net	3	2	6	0	4	0	3	0	0	0	5/10
testasp.vulnweb.com	1	2	0	0	1	0	1	0	0	0	4/10

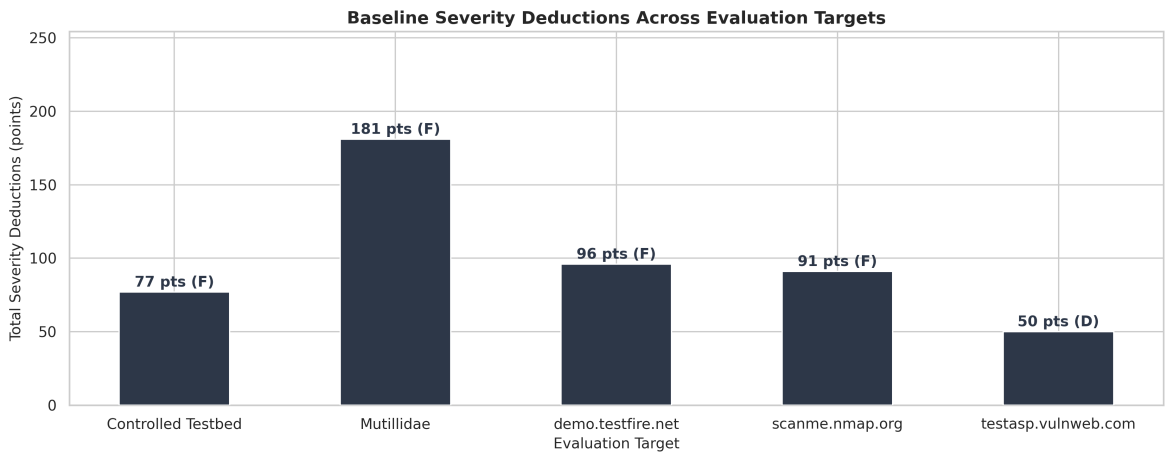


Figure 4.1: Severity deduction scores across all five baseline scans, illustrating the range from 50 (testasp.vulnweb.com) to 181 (Mutillidae).



Figure 4.2: Severity distribution by target, showing the breakdown of Critical, High, Medium, Low and Informational findings across all five baseline scans.

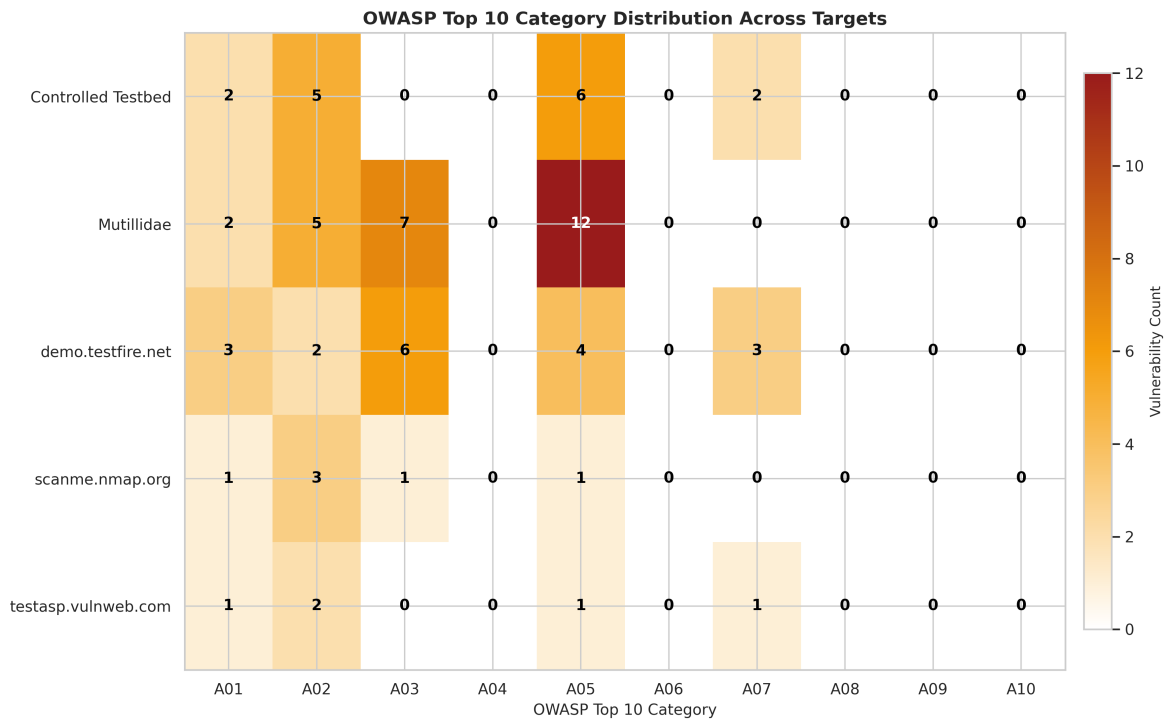


Figure 4.3: OWASP Top 10 category heatmap across all five baseline scans, with cell values indicating the number of vulnerabilities mapped to each category.

The baseline scans consistently detected vulnerabilities across four of the ten OWASP Top 10 categories: A01 (Broken Access Control), A02 (Cryptographic Failures), A03 (Injection) and A05 (Security Misconfiguration). The remaining six categories received zero detections across four of the five targets. This pattern reflects a structural constraint of the scanning layer rather than a deficiency in the classification pipeline: the system operates at the network level via Nmap, and the four detected categories align with what service enumeration, NSE scripting and header analysis can observe. The exception is demo.testfire.net, which achieved 5/10 coverage by additionally triggering A07 (Identification and Authentication Failures) detections, a result consistent with its financial-application design exposing authentication endpoints that Nmap’s NSE scripts can partially enumerate. Categories such as A04 (Insecure Design), A08 (Software and Data Integrity Failures) and A09 (Security Logging and Monitoring Failures) are architectural or operational concerns that no automated network-layer scanner can detect. The 40% OWASP coverage figure across four of five targets should be read as the system reporting what it can detect rather than inflating coverage beyond its scanner’s reach.

4.3 Phase 2: Contextual Weighting Comparative Analysis

Table 4.4: Phase 2 contextual weighting comparison on the Mutillidae target.

Configuration	Total De- ductions	Grade	Notes
contextualWeighting OFF (flat)	107	F	Flat severity scoring only
All toggles ON (contextual)	181	F	Contextual multipliers active
Amplification	$(181 - 107) \div 107 = +69.2\%$		

Contextual weighting produced a 69.2% amplification in total severity deductions on the Mutillidae baseline, raising the score from 107 (flat) to 181 (contextual), as shown in Table 4.4 and Figure 4.4. Port exposure on web-facing services is the dominant contextual factor: the `contextualWeighting` module identified Mutillidae’s HTTP and HTTPS endpoints as high-exposure services warranting a 1.5× multiplier; Figure 4.5 confirms that the service exposure and authentication weakness channels both remained at 1.0× throughout, meaning those two multiplier dimensions had no measurable effect on this target.

The combined 1.5× multiplier applied to approximately 60% of the vulnerability set accounts for the bulk of the 74-point differential, confirming that the weighting system meaningfully differentiates risk profiles beyond flat CVSS-style severity assignment; the inactive channels limit conclusions about the full multiplier model from this target alone.

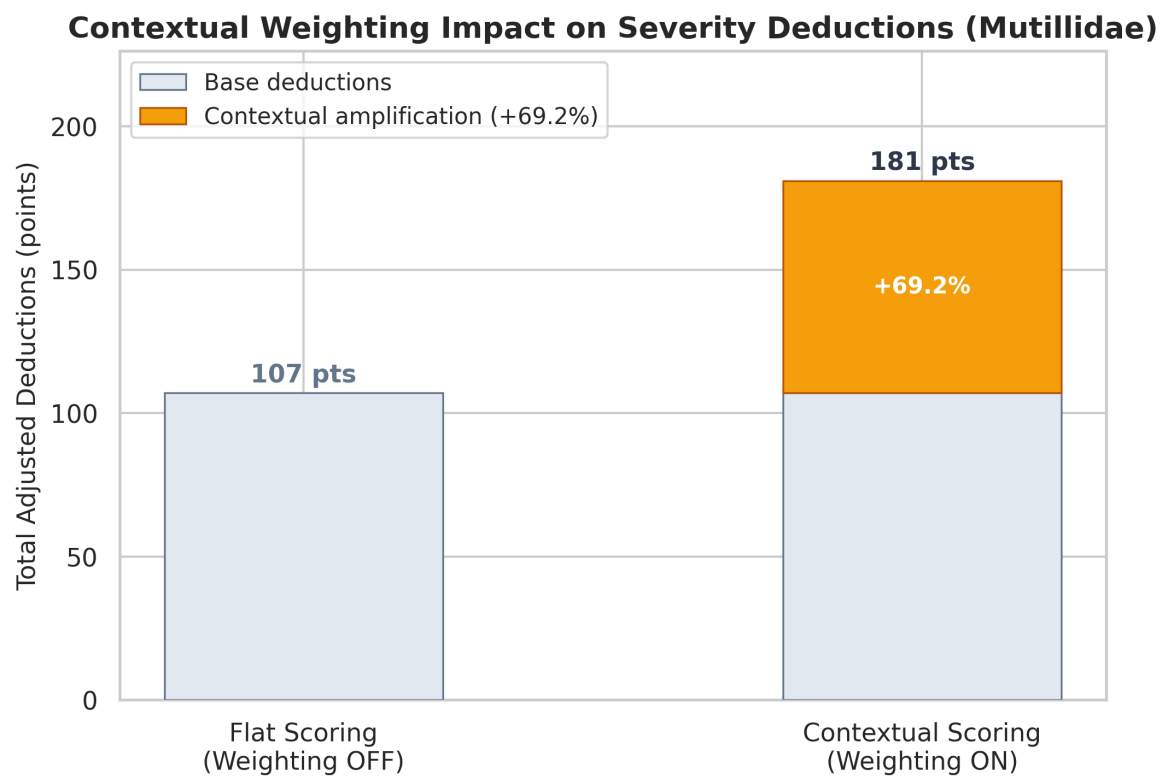


Figure 4.4: Comparison of flat (weighting OFF) versus contextual (weighting ON) severity deductions on the Mutillidae target, showing the 69.2% amplification produced by the contextual multiplier system.

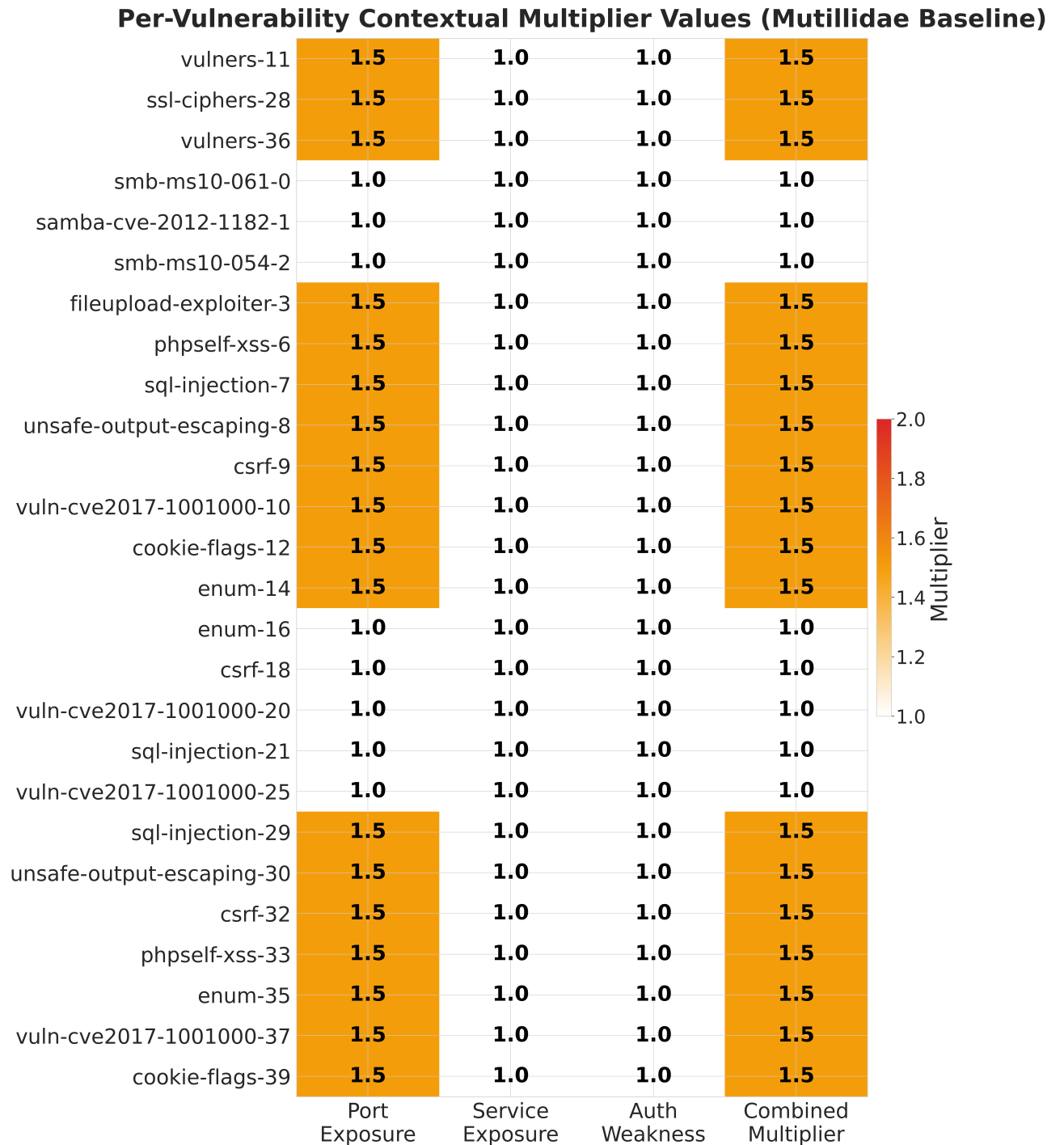


Figure 4.5: Per-vulnerability contextual multiplier values for the Mutillidae baseline scan, showing port exposure, service exposure, authentication weakness and combined multiplier across all 26 findings.

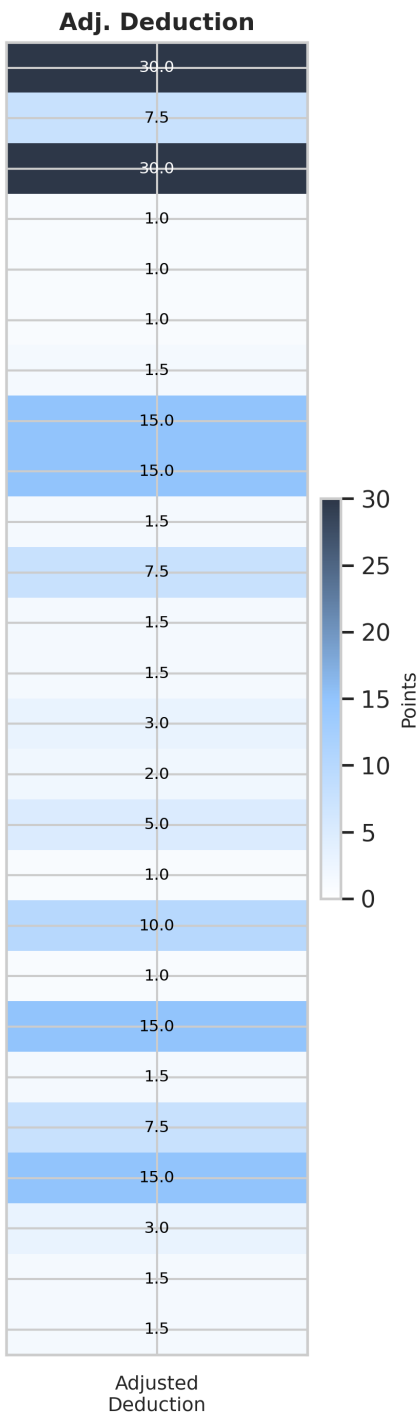


Figure 4.6: Adjusted deduction scores for each vulnerability in the Mutillidae baseline scan, corresponding to the multiplier values in Figure 4.5.

4.4 Phase 3: Ablation Testing

Table 4.5: Phase 3 ablation test results on the Mutillidae target. Each row disables a single pipeline component while all others remain active.

Configuration	Sev. Ded.	Δ vs Full	Observed Effect
Full pipeline (baseline)	181	—	All components active
<code>contextualWeighting</code> OFF	107	−74	Port exposure multipliers removed
<code>owaspMapping</code> OFF	166	−15	All vulns collapse to A05 fallback
<code>aiAnalysis</code> OFF	181	0	Remediation quality removed, score unchanged
<code>hallucinationGuard</code> OFF	165	−16	Validation layer removed

Disabling `contextualWeighting` produced the largest single-component delta at −74 points (181 to 107), confirming contextual weighting as the highest-impact component and showing the contextual layer accounts for approximately 41% of total severity deductions in the full configuration. The design as a distinct, toggleable module means operators who prefer flat scoring can disable the multiplier system without affecting the remainder of the pipeline.

Disabling `owaspMapping` reduced deductions by 15 points as all vulnerabilities collapsed to the A05 fallback, eliminating the category breadth penalty the scoring engine applies when findings span multiple OWASP categories. The modest reduction confirms the breadth penalty contributes a secondary but measurable effect, and that the fallback assigns a conservative default rather than discarding unmapped findings.

Disabling `aiAnalysis` produced a zero-point delta, as the AI component contributes remediation depth rather than numerical scoring; the +10 bonus affects grade interpretation rather than the severity deduction total. Disabling `hallucinationGuard` reduced deductions by 16 points, a contribution comparable to the OWASP breadth penalty and consistent with the guard’s role as a validation layer applying confidence-weighted adjustments to AI classifications rather than functioning as a primary scoring mechanism.

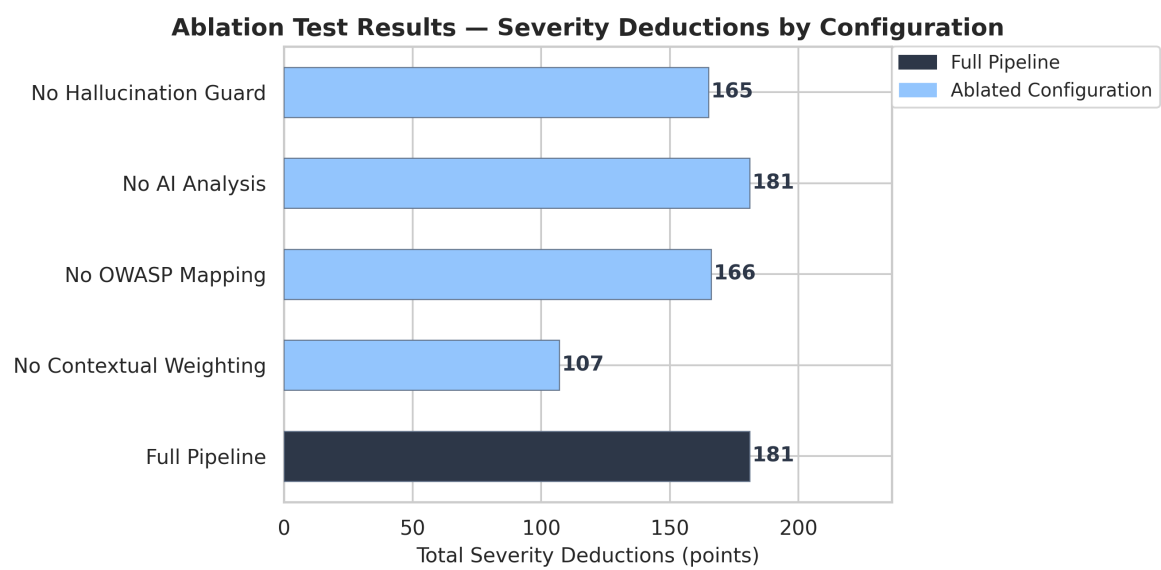


Figure 4.7: Ablation test severity deductions on the Mutillidae target, comparing the full pipeline against each single-component removal configuration.

4.5 Phase 4: Lifecycle Delta Demonstration

Phase 4 was conducted exclusively on the controlled testbed (127.0.0.1:8080), which is the only target whose vulnerability configuration permits selective remediation between scan cycles. The baseline scan was followed by targeted remediation of eight vulnerabilities, after which a rescan was performed and the delta comparison module generated the results presented in Table 4.6.

Table 4.6: Phase 4 lifecycle delta comparison on the controlled testbed, showing before (baseline) and after (remediated) scan results.

Scan	Score	Total Vulns	Critical	High	Med.	Low
Before (Baseline)	3	15	0	2	3	2
After (Remediated)	60	7	0	1	1	1
Delta	+57	−8	0	−1	−2	−1

The score improved by 57 points, from 3 to 60, following remediation of 8 of 15 vulnerabilities, representing a 53% reduction in total findings as shown in Figure 4.8. The post-remediation scan correctly identified all 7 remaining findings as persisting vulnerabilities not targeted in this cycle, demonstrating that the delta comparison

module accurately distinguishes resolved from persisting findings. The residual 1 High and 1 Medium findings held the post-remediation score at 60, confirming that persisting high-severity vulnerabilities are penalised proportionally as designed.

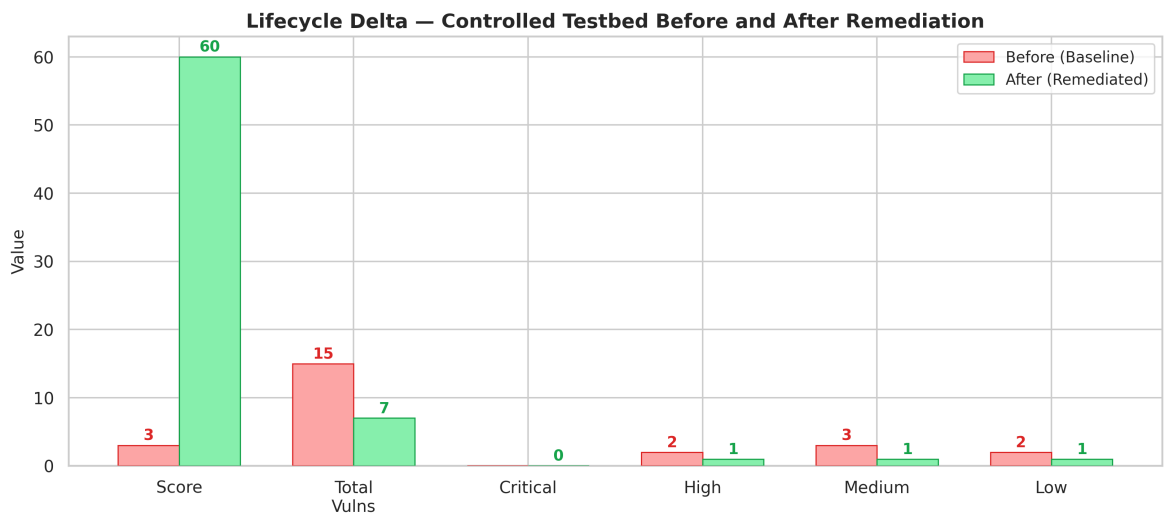


Figure 4.8: Lifecycle delta comparison between the baseline and remediated scans on the controlled testbed, showing the score improvement from 3 to 60 and the reduction from 15 to 7 total vulnerabilities.

4.6 Phase 5: Longitudinal Hallucination Reliability

Table 4.7: Longitudinal hallucination guard metrics across all evaluation scans where both `aiAnalysis` and `hallucinationGuard` were enabled.

Scan	Target / Configuration	Trust Score	Fab. CVEs	Notes
1	Mutillidae (baseline)	~55	0	Below threshold
2	Mutillidae (No OWASP)	0	0	Guard detects total OWASP disagreement
3	Controlled Testbed (baseline)	~50	0	Below threshold
4	Controlled Testbed (Phase 2)	~47	0	Below threshold
5	Controlled Testbed (No OWASP)	~40	0	Below threshold
6	scanme.nmap.org (baseline)	~72	0	Above threshold
7	scanme.nmap.org (Phase 2)	~71	0	Above threshold
8	testasp.vulnweb.com (baseline)	~85	0	Above threshold
9	testasp.vulnweb.com (Phase 2)	~85	0	Above threshold
10	demo.testfire.net (baseline)	~38	0	Below threshold
11	demo.testfire.net (Phase 2)	~38	0	Below threshold

Zero fabricated CVEs were recorded across all 11 scans in Table 4.7, confirming that the AI model produced no invented vulnerability identifiers in any tested configuration. Trust scores varied from 0 on the Mutillidae “No OWASP” configuration (expected: with the mapper disabled, the guard correctly detects near-total OWASP disagreement) to approximately 85 on `testasp.vulnweb.com`; only `scanme.nmap.org` and `testasp.vulnweb.com` consistently exceeded the 70-point threshold, as shown in Figure 4.9. Figure 4.10 shows Mutillidae accumulating the highest risk flag count at approximately 18, consistent with its larger vulnerability set producing more opportunities for AI classification disagreements.

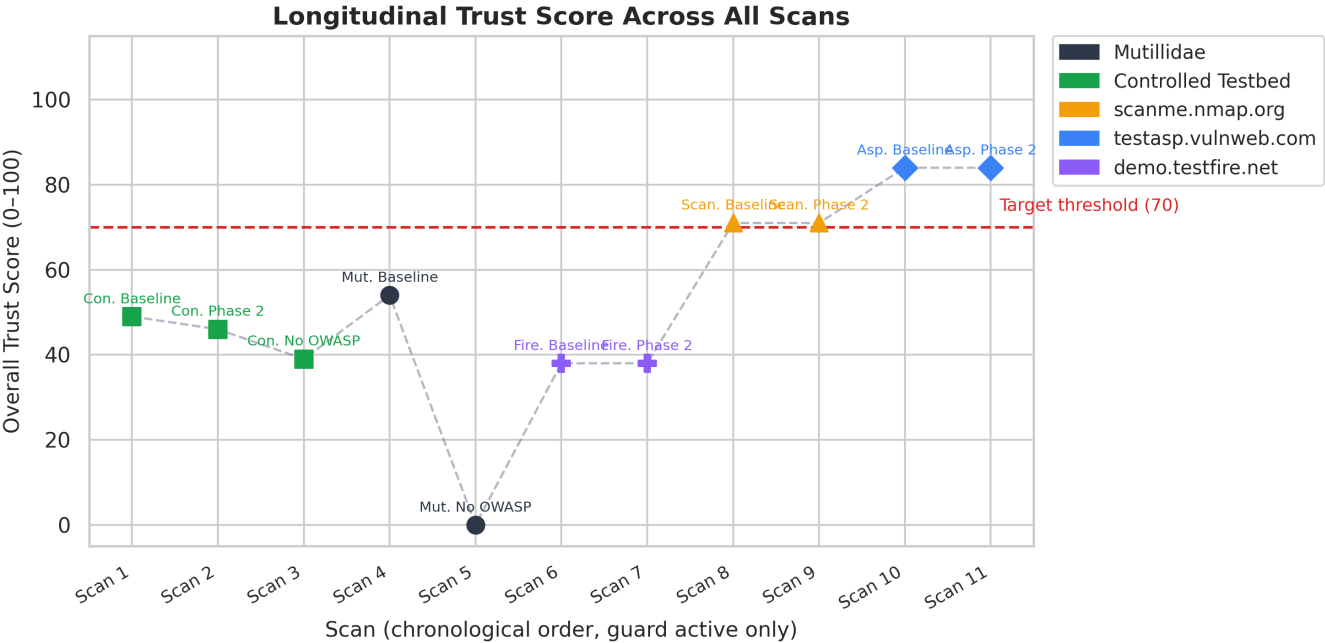


Figure 4.9: Trust scores across all evaluation scans, with the 70-point threshold indicated. Only scanme.nmap.org and testasp.vulnweb.com consistently exceed the threshold.

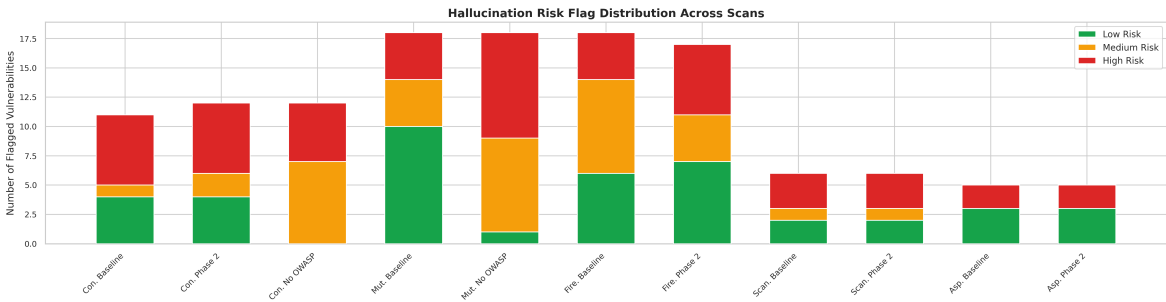


Figure 4.10: Hallucination risk flag breakdown per scan, showing stacked Low, Medium and High risk flags. Mutillidae baseline exhibits the highest total flag count at approximately 18 flags.

The 70-point target was not met on three of the five evaluation targets, an outcome that reflects genuine AI classification inconsistencies rather than a systematic deficiency in the guard itself. The consistently low scores on demo.testfire.net (approximately 38) indicate higher disagreement rates on the IIS/ASP.NET stack; the variation across targets confirms that trust score is sensitive to target complexity and technology stack, and that the threshold may require calibration for non-PHP environments. Only the port exposure multiplier channel activated across all five targets, with service exposure and authentication weakness channels remaining at 1.0× throughout.

4.7 Tool Capability Comparison

Table 4.8 positions PurpleTeam Suite against six representative tools, measuring integration completeness across the full assessment lifecycle rather than raw detection performance.

Table 4.8: Feature comparison across representative vulnerability assessment tools. Integration completeness is assessed rather than raw detection performance.

Capability	PurpleTeam	Nmap	OWASP ZAP	OpenVAS	Burp Suite	Nessus	Splunk SOAR
OWASP Top 10 classification	✓	✗	✓	Partial	✓	Partial	Partial
Deterministic risk scoring	✓	✗	✓	✓	✓	✓	✓
Contextual weighting	✓	✗	✗	✗	✗	✗	Partial
AI remediation guidance	✓	✗	✗	✗	Partial	Partial	✓
Hallucination guard	✓	✗	✗	✗	✗	✗	✗
Iterative delta comparison	✓	✗	✗	Partial	✗	Partial	✓
Exportable PDF reporting	✓	Partial	✓	✓	✓	✓	✓
Open-source / no licence cost	✓	✓	✓	✓	✗	✗	✗

PurpleTeam Suite is the only tool in the comparison implementing a closed-loop pipeline combining network-layer scanning, AI-enriched classification, deterministic hallucination validation, contextual scoring and iterative lifecycle comparison in a single application; Splunk SOAR comes closest but operates as an orchestration layer over existing scanners and carries a commercial licence cost. The comparison is bounded by the scanner-layer scope, as OWASP ZAP and Burp Suite reach application-layer vulnerability classes that network-level scanning cannot detect.

4.8 Summary

The five-phase evaluation produced quantitative evidence across all pipeline components: baseline scores from 50 to 181 (Phase 1), a 69.2% contextual weighting amplification (Phase 2), a 74-point ablation delta confirming contextual weighting as the highest-impact component (Phase 3), a 57-point lifecycle improvement with 53% vulnerability reduction (Phase 4), and zero fabricated CVEs across 11 scans with trust scores sensitive

to target complexity (Phase 5). These results inform the conclusions in Chapter 5.

5. Conclusions

5.1 Introduction

Large language models are, by design, probabilistic systems: they generate outputs by sampling from a learned distribution rather than following deterministic rules, which means that identical inputs can produce different outputs across runs and that confident-sounding claims can be factually ungrounded. In a security assessment context, this is an accuracy concern and an operational risk, because a remediation recommendation that is plausibly worded but factually incorrect does not fail silently, it actively misleads the analyst acting on it. The central engineering challenge this project addressed was therefore not simply whether AI could assist vulnerability assessment, but whether a pipeline architecture could impose sufficient deterministic constraint on a probabilistic model to make its outputs operationally trustworthy, without discarding the explanatory and summarisation value that made AI assistance worth integrating in the first place.

This project aimed to design, implement and empirically evaluate a deterministic-first, AI-assisted purple-team framework integrating vulnerability discovery, exploitability-aware prioritisation, structured remediation guidance, hallucination mitigation and iterative reassessment within a unified lifecycle model. The approach taken was to position deterministic analysis upstream of AI assistance at every stage: a keyword-matched OWASP classifier establishes ground-truth classifications before the model sees the data; a structured JSON schema and low-temperature configuration constrain what the model can produce; and a post-AI hallucination guard cross-validates model output against the deterministic evidence channel, surfacing disagreements as quantified trust scores rather than treating AI confidence as self-validating. Three research questions from Section 1.2 guided the evaluation: RQ I asked whether this framework could integrate the full assessment lifecycle within a single reproducible pipeline; RQ II asked how consistently the hybrid classification mechanism could map findings to OWASP Top 10 categories across heterogeneous targets; and RQ III asked whether structured hallucination mitigation through deterministic cross-validation could measurably constrain AI-generated inconsistencies. The following sections answer each question with evidence from Chapter 4, identify contributions relative to the gaps established in Chapter 2, reflect honestly on limitations, and propose concrete future work.

5.2 Answering the Research Questions

RQ I: Lifecycle Integration

RQ I asked whether the framework could integrate vulnerability discovery, prioritisation and remediation guidance within a single reproducible pipeline. The answer is affirmative, with a qualification on scanning scope. PurpleTeam Suite chains Nmap-based discovery, deterministic OWASP classification, contextual scoring, AI analysis, hallucination validation, delta comparison and PDF reporting without manual intervention between stages. Phase 4 (Section 4.5) demonstrated the lifecycle end-to-end: the scan-remediate-rescan cycle on the controlled testbed produced a 57-point score improvement (3 to 60) and a 53% reduction in total findings (15 to 7), with the delta comparison module correctly classifying all resolved and persisting findings. Table 4.8 confirms that PurpleTeam Suite is the only tool in the comparison implementing a closed-loop pipeline of this kind in a single deployable application. The qualification is that integration is bounded by Nmap’s network-layer scope, with four of the ten OWASP categories structurally beyond the scanner’s reach, a constraint acknowledged by the system’s reporting rather than obscured.

RQ II: Classification Consistency

RQ II asked how consistently the hybrid classification mechanism could map findings to OWASP Top 10 categories across heterogeneous authorised targets. The answer is a qualified positive. Four of the five evaluation targets achieved consistent 4/10 OWASP category coverage (A01, A02, A03, A05) across all Phase 1 and Phase 2 scans; demo.testfire.net achieved 5/10 (Section 4.2, Table 4.3). The deterministic keyword mapper’s fixed classification logic produces the same OWASP output for the same scan data on every run, a reproducibility property that probabilistic classifiers do not provide. Phase 3 ablation (Section 4.4) confirmed the mapper as the ground-truth layer: disabling it collapsed all findings to the A05 fallback, reducing deductions by 15 points and eliminating the category breadth penalty. The 4/10 ceiling reflects the network-layer scope constraint, not a classification failure; A04, A08 and A09 are architectural concerns beyond any automated network-layer scanner.

RQ III: Hallucination Mitigation

RQ III asked whether structured hallucination mitigation through deterministic cross-validation could measurably constrain AI-generated inconsistencies and fabricated vulnerability references. The answer is a qualified positive. Zero fabricated CVE identifiers were recorded across all 11 AI-guarded scans (Section 4.6, Table 4.7), the

strongest result on the most operationally damaging failure mode. The guard’s weighted trust formula produced scores from 0 (Mutillidae, mapper disabled) to approximately 85 (testasp.vulnweb.com); only scanme.nmap.org and testasp.vulnweb.com consistently exceeded the 70-point threshold, while the remaining three targets produced scores between 38 and 55. The variation reflects genuine AI classification inconsistency driven by target complexity and technology stack: low scores on the IIS/ASP.NET target indicate higher OWASP disagreement rates on that stack, and the 70-point threshold may require recalibration across technology profiles rather than being treated as universal.

5.3 Contributions

Measured against the integration gaps identified in Section 2, this project makes several contributions. It delivers an integrated pipeline closing the scan, classify, score, AI-explain, validate and re-scan loop in one deployable tool, a combination no reviewed prior work achieves in a single application. It operationalises the direction toward scanner-grounded hallucination control, pointed toward by Hammar et al. and Farquhar et al., by cross-validating AI output against deterministic scan evidence rather than assessing surface plausibility, producing zero fabricated CVEs across all tested configurations. The exploitability-aware contextual weighting module (Objective G) is ablation-tested: Phase 2 and Phase 3 together show a 69.2% amplification on Mutillidae and a 74-point ablation delta, confirming a measurable contribution to risk differentiation beyond flat CVSS-style severity assignment. The runtime feature toggle system enabling per-component ablation without codebase modification provides the methodological infrastructure for the Phase 3 evaluation and a reusable pattern for controlled experimentation within operational security tooling.

5.4 Limitations

Several limitations bound the conclusions. The evaluation corpus of five authorised targets limits generalisability across diverse deployment environments and technology stacks. The AI component depends on a single LLM provider; behaviour may differ across models or versions, a dependency the results cannot disentangle from the guard’s performance. Phase 3 ablation was conducted on Mutillidae only, so per-component contribution estimates are derived from a single technology profile. Three of the five targets fell below the 70-point trust threshold, an outcome that could reflect prompt calibration needs for non-PHP stacks or threshold miscalibration rather than systematic guard failure. The contextual multipliers are researcher-defined heuristics: only the port-exposure channel activated across all five targets, leaving the service-exposure and

authentication-weakness channels unexercised and unvalidated against empirical exploit-frequency data. The network-layer scanning scope structurally limits OWASP coverage to 4 of 10 categories, with A04, A08 and A09 beyond automated reach regardless of classification quality.

5.5 Recommendations and Future Work

Multi-provider LLM comparison (covering Claude, GPT-class models and local open-weights alternatives) would separate trust score behaviour from single-model idiosyncrasy and test whether the zero fabricated CVE result holds across providers. Prompt engineering refinement targeting IIS/ASP.NET stacks is a direct response to the low trust scores on `demo.testfire.net` and `testasp.vulnweb.com`. Trust threshold recalibration against a larger and more diverse target corpus would establish whether 70 points is a sound general threshold or requires per-technology-profile setting. Empirical multiplier calibration using exploit-prediction data such as EPSS would replace researcher-defined heuristics with frequency-grounded weights. Coverage extension through OWASP ZAP or Burp Suite integration would reach application-layer vulnerability classes outside Nmap's scope, directly addressing the 4/10 OWASP ceiling. Expanding the ablation methodology across all five evaluation targets, rather than Mutillidae alone, would produce more generalisable per-component contribution estimates.

The project demonstrates that deterministic-first integration with constrained AI assistance is a viable architecture for bounded-risk automated vulnerability assessment, and that the most operationally damaging hallucination failure mode can be controlled through scanner-evidence cross-validation even where overall trust scores remain moderate.

Bibliography

- [1] K. Abdulghaffar, N. Elmrabit, and M. Yousefi, “Enhancing web application security through automated penetration testing with multiple vulnerability scanners,” *Computers*, vol. 12, no. 11, p. 235, Nov. 2023. DOI: [10.3390/computers12110235](https://doi.org/10.3390/computers12110235)
- [2] B. A. Alahmadi, L. Axon, and I. Martinovic, “99% false positives: A qualitative study of SOC analysts’ perspectives on security alarms,” in *USENIX Security Symposium*, 2022.
- [3] M. Althunayyan, N. Saxena, S. Li, and P. Gope, “Evaluation of black-box web application security scanners in detecting injection vulnerabilities,” *Electronics*, vol. 11, no. 13, p. 2049, Jun. 2022. DOI: [10.3390/electronics11132049](https://doi.org/10.3390/electronics11132049)
- [4] G. Bhandari, N. Gavric, and A. Shalaginov, “Generating vulnerability security fixes with code language models,” *Information and Software Technology*, vol. 185, Sep. 2025. DOI: [10.1016/j.infsof.2025.107786](https://doi.org/10.1016/j.infsof.2025.107786)
- [5] A. Bhattacharya, T. Ramachandran, S. Banik, C. P. Dowling, and S. D. Bopardikar, “Automated adversary emulation for cyber-physical systems via reinforcement learning,” in *Proceedings – 2020 IEEE International Conference on Intelligence and Security Informatics (ISI 2020)*, Accessed: Mar. 02, 2026, Nov. 2020. [Online]. Available: <http://arxiv.org/abs/2011.04635>
- [6] R. A. Bridges et al., “Testing SOAR tools in use,” *Computers & Security*, vol. 129, p. 103201, Jun. 2023. DOI: [10.1016/j.cose.2023.103201](https://doi.org/10.1016/j.cose.2023.103201)
- [7] I. Chalvatzis, D. A. Karras, and R. C. Papademetriou, “Evaluation of security vulnerability scanners for small and medium enterprises business networks resilience towards risk assessment,” in *Proceedings of 2019 IEEE International Conference on Artificial Intelligence and Computer Applications (ICAICA 2019)*, Mar. 2019, pp. 52–58. DOI: [10.1109/ICAICA.2019.8873438](https://doi.org/10.1109/ICAICA.2019.8873438)
- [8] C. Chen et al., “When ChatGPT meets smart contract vulnerability detection: How far are we?” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 4, Aug. 2024. DOI: [10.1145/3702973](https://doi.org/10.1145/3702973)
- [9] Y. Dong et al., “Safeguarding large language models: A survey,” *Artificial Intelligence Review*, vol. 58, no. 12, Dec. 2025. DOI: [10.1007/s10462-025-11389-2](https://doi.org/10.1007/s10462-025-11389-2)
- [10] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, “Detecting hallucinations in large language models using semantic entropy,” *Nature*, vol. 630, no. 8017, pp. 625–630, Jun. 2024. DOI: [10.1038/s41586-024-07421-0](https://doi.org/10.1038/s41586-024-07421-0)

- [11] K. Hammar, T. Alpcan, and E. C. Lupu, *Hallucination-resistant security planning with a large language model*, Accessed: Mar. 01, 2026, Feb. 2026. [Online]. Available: <https://arxiv.org/pdf/2602.05279v1>
- [12] H. Holm and T. Sommestad, "Realistic and balanced automated threat emulation," *Computers & Security*, vol. 151, p. 104351, Apr. 2025. DOI: [10.1016/j.cose.2025.104351](https://doi.org/10.1016/j.cose.2025.104351)
- [13] H. Howland, "CVSS: Ubiquitous and broken," *Digital Threats: Research and Practice*, vol. 4, no. 1, Mar. 2023. DOI: [10.1145/3491263](https://doi.org/10.1145/3491263)
- [14] J. Jacobs, S. Romanosky, O. Suciu, B. Edwards, and A. Sarabi, "Enhancing vulnerability prioritization: Data-driven exploit predictions with community-driven insights," in *Proceedings – 8th IEEE European Symposium on Security and Privacy Workshops (Euro S&PW 2023)*, Accessed: Feb. 28, 2026, Jun. 2023, pp. 194–206. [Online]. Available: <http://arxiv.org/abs/2302.14172>
- [15] S. Kansal and Prasad, "AI-driven vulnerability detection and resolution frameworks for enhanced security posture," *Integrated Journal for Research in Arts and Humanities*, vol. 4, no. 6, pp. 444–458, 2024, Accessed: Feb. 02, 2026. [Online]. Available: <https://www.ijrah.com/index.php/ijrah/article/view/666>
- [16] L. Kersten, K. Beelen, E. Zambon, C. Snijders, and L. Allodi, "A field study to uncover and a tool to support the alert investigation process of tier-1 analysts," in *Symposium on Usable Security and Privacy (USEC)*, Accessed: Feb. 28, 2026, NDSS Symposium, 2025. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/auto-draft-595/>
- [17] P. Khandagale, A. Karande, P. Gidde, and R. Narkhede, "A comprehensive review of AI-driven vulnerability detection and patch recommendation systems," *International Journal of Engineering Development and Research*, vol. 13, no. 1, pp. 120–125, 2025, Accessed: Feb. 02, 2026. [Online]. Available: <https://rjwave.org/ijedr/viewpaperforall.php?paper=IJEDR2501013>
- [18] V. Koscinski, M. Nelson, A. Okutan, R. Falso, and M. Mirakhorli, *Conflicting scores, confusing signals: An empirical study of vulnerability scoring systems*, Accessed: Feb. 28, 2026, Aug. 2025. [Online]. Available: <http://arxiv.org/abs/2508.13644>
- [19] M. Landauer, W. Hotwagner, T. Boenke, F. Skopik, and M. Wurzenberger, *AttackMate: Realistic emulation and automation of cyber attack scenarios across the kill chain*, Accessed: Mar. 02, 2026, Jan. 2026. [Online]. Available: <http://arxiv.org/abs/2601.14108>

- [20] T. H. M. Le, H. Chen, and M. A. Babar, “A survey on data-driven software vulnerability assessment and prioritization,” *ACM Computing Surveys*, vol. 55, no. 5, Apr. 2022. DOI: [10.1145/3529757](https://doi.org/10.1145/3529757)
- [21] M. Lei, T. A. Madi, M. Nitschke, L. Zhao, and M. Pourzandi, “Multi-target risk score aggregation for security evaluation of network environments,” in *Proceedings of the International Conference on Cloud Computing Technology and Science (CloudCom)*, 2024, pp. 71–78. DOI: [10.1109/CloudCom62794.2024.00023](https://doi.org/10.1109/CloudCom62794.2024.00023)
- [22] J. Lin and D. Mohaisen, “From large to mammoth: A comparative evaluation of large language models in zero-shot vulnerability detection,” in *Network and Distributed System Security Symposium (NDSS)*, Mar. 2025. DOI: [10.14722/ndss.2025.241491](https://doi.org/10.14722/ndss.2025.241491)
- [23] Y. Luo et al., *HALURust: Exploiting hallucinations of large language models to detect vulnerabilities in Rust*, Accessed: Mar. 01, 2026, Mar. 2025. [Online]. Available: <http://arxiv.org/abs/2503.10793>
- [24] F. M. Mateo-Tudela, J. R. B. Robles-Higuera, J. B. Blanco-Higuera, J. A. Sánchez-Montalvo, and M. I. Argyros, “On combining static, dynamic and interactive analysis security testing tools to improve OWASP top ten security vulnerability detection in web applications,” *Applied Sciences*, vol. 10, no. 24, p. 9119, Dec. 2020. DOI: [10.3390/app10249119](https://doi.org/10.3390/app10249119)
- [25] J. G. Oakley, “Purple teaming,” in *Professional Red Teaming*, 2019, pp. 105–115. DOI: [10.1007/978-1-4842-4309-1_8](https://doi.org/10.1007/978-1-4842-4309-1_8)
- [26] M. Pan, P. Wu, Y. Zou, C. Ruan, and T. Zhang, “An automatic vulnerability classification framework based on BiGRU-TextCNN,” *Procedia Computer Science*, vol. 222, pp. 377–386, 2023. DOI: [10.1016/j.procs.2023.08.176](https://doi.org/10.1016/j.procs.2023.08.176)
- [27] V. Rawte et al., “The troubling emergence of hallucination in large language models — an extensive definition, quantification, and prescriptive remediations,” in *EMNLP 2023 – 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 2541–2573. DOI: [10.18653/v1/2023.emnlp-main.155](https://doi.org/10.18653/v1/2023.emnlp-main.155)
- [28] J. M. Redondo and D. Cuesta, “Towards improving productivity in Nmap security audits,” *Journal of Web Engineering*, vol. 18, no. 7, pp. 539–578, Nov. 2019. DOI: [10.13052/jwe1540-9589.1871](https://doi.org/10.13052/jwe1540-9589.1871)
- [29] P. Sane, “Is the OWASP top 10 list comprehensive enough for writing secure code?” In *ACM International Conference Proceeding Series*, May 2020, pp. 58–61. DOI: [10.1145/3437075.3437089](https://doi.org/10.1145/3437075.3437089)
- [30] A. Sarkar and I. Drosos, *Vibe coding: Programming through conversation with artificial intelligence*, Online, Accessed: Feb. 06, 2026, Oct. 2025. [Online]. Available: <http://arxiv.org/abs/2506.23253>

- [31] A. Singh, E. S. Sharma, B. K. Reddy, P. Soni, S. S. Ghuman, and U. S. Gill, “Automated network vulnerability assessment with Nmap: A comprehensive approach,” in *2024 Second International Conference on Advanced Computing & Communication Technologies (ICACCTech)*, 2024, pp. 208–214. DOI: [10.1109/ICACCTech65084.2024.00043](https://doi.org/10.1109/ICACCTech65084.2024.00043)
- [32] J. Vandeleur, *Continuous purple teaming: A practical approach*, SANS Institute Blog, Accessed: Feb. 01, 2026. [Online]. Available: <https://www.sans.org/blog/continuous-purple-teaming-practical-approach-strengthening-offensive-capabilities>
- [33] S. F. Wen and B. Katt, “A quantitative security evaluation and analysis model for web applications based on OWASP application security verification standard,” *Computers & Security*, vol. 135, p. 103532, Dec. 2023. DOI: [10.1016/j.cose.2023.103532](https://doi.org/10.1016/j.cose.2023.103532)
- [34] J. Xu, Y. Fu, S. H. Tan, and P. He, “Aligning the objective of LLM-based program repair,” in *Proceedings – International Conference on Software Engineering*, Accessed: Mar. 01, 2026, Feb. 2025, pp. 2548–2560. [Online]. Available: <http://arxiv.org/abs/2404.08877>

Appendices

A. Supervisor Meeting Log

Table A.1 summarises the supervisory meetings held with David Neilson over the course of the project. The module requires a minimum of four meetings during the specified review windows, all of which were attended. The majority of sessions focused on incremental refinements to the report, including formatting corrections, word choice adjustments and figure scaling, as L^AT_EX was adopted as the typesetting tool and required a period of familiarisation. Where substantive technical or structural feedback was provided, this is noted in the discussion summary.

Table A.1: Supervisor meeting log with David Neilson.

Meeting	Date	Type	Discussion Summary
1	17 Feb 2026	Proposal Review	Reviewed the project proposal and ethical approval form. David confirmed the scope and aim were appropriate for the module. Minor wording adjustments were suggested to tighten the objectives, and the Gantt chart layout was discussed to ensure all twelve weeks and key deadlines were represented.
2	03 Mar 2026	Report Review 1	First review of the literature review chapter. Feedback centred on reducing the use of overly formal or pretentious phrasing in several paragraphs, alongside corrections to figure scaling where images were either too large or clipped at the margins. Some citation formatting inconsistencies were also flagged for correction.
3	24 Mar 2026	Report Review 2	Second report review covering Chapters 2 and 3 in their current state. David noted that certain figures still needed resizing and that a small number of captions lacked sufficient descriptive detail. Additional feedback included replacing instances of the term “software artefact” when referring to the application itself, as this was considered too vague for the context. The overall structure and progression of the design chapter were confirmed as sound.
4	07 Apr 2026	Report Review 3	Final scheduled review session covering Chapter 4 alongside the updated earlier chapters. Feedback was largely editorial, including further word choice refinements and ensuring consistent voice throughout the report. David confirmed the evaluation approach and the framing of the ablation study were appropriate, and highlighted the need to keep the word count within the prescribed limits as several sections were running over target.

B. Repository Information and Statistics

The full source code for PurpleTeam Suite is hosted on GitHub at: <https://github.com/umfhero/PurpleTeamAI>

As the application is built on the Electron framework, it can be compiled into a standalone executable for Windows, macOS, and Linux. Testing was conducted on Windows and Kali Linux. The compiled .exe installer for Windows and equivalent binaries for other platforms can be produced from the repository source using the standard Electron build toolchain.

Cross-Platform Build Verification

Figures B.1 and B.2 confirm that PurpleTeam Suite compiles and runs natively on Kali Linux and macOS respectively, consistent with the cross-platform requirement described in Appendix C.

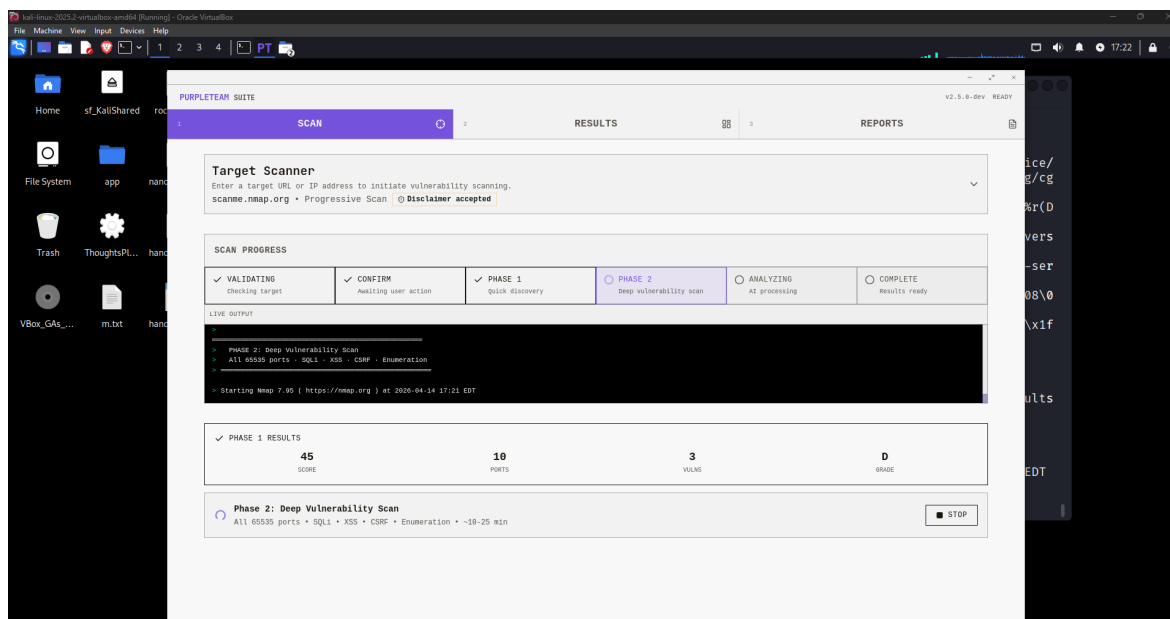


Figure B.1: PurpleTeam Suite running on Kali Linux.

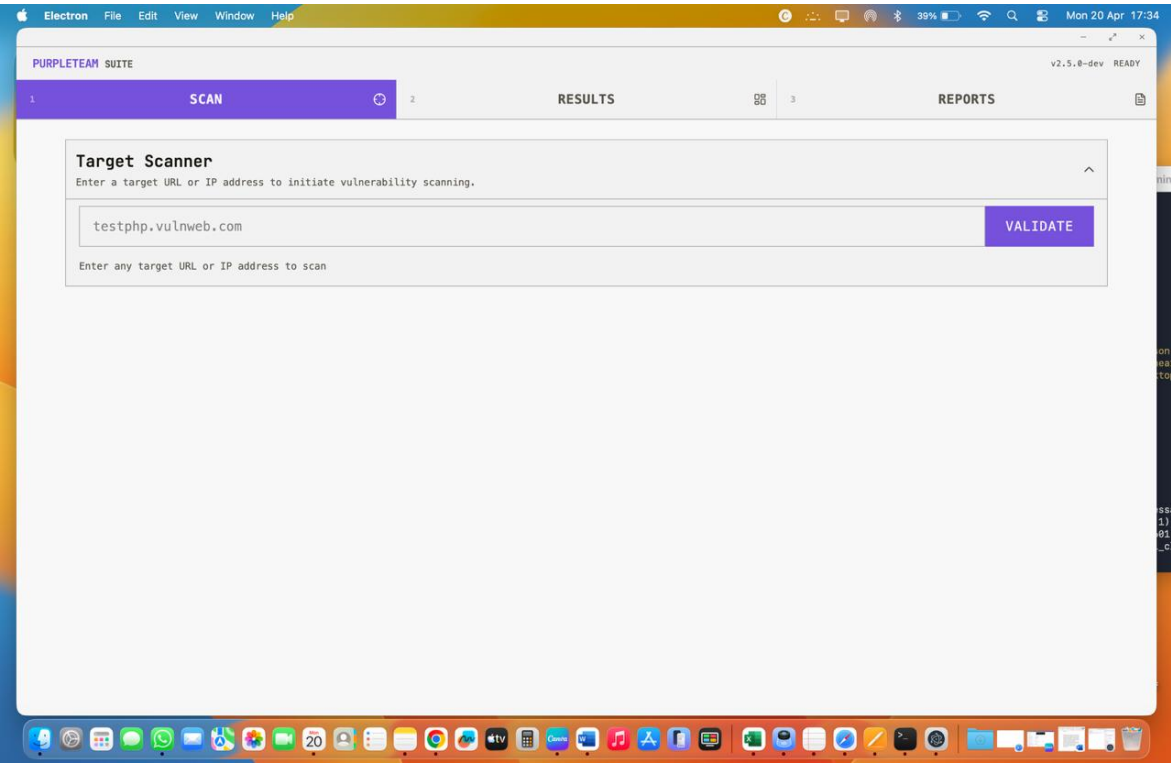


Figure B.2: PurpleTeam Suite running on macOS.

Table B.1: Repository statistics at time of submission.

Metric	Value
Total source files (<code>.ts/.tsx</code> , excl. <code>node_modules</code>)	43
Lines of TypeScript (estimated)	~10,500
Pipeline modules	10 (scanner, parser, OWASP mapper, scorer, LLM, hallucination guard, delta, reports, feature toggles, metrics)
Evaluation reports exported	11 PDF files (<code>data/reports/</code>)

C. Project Schedule



Figure C.1: Planned versus actual project schedule.

D. Implementation Environment Details

The system was developed using the technology stack described in Table 3.2 in Section 3.2.7. Selection criteria prioritised cross-platform compatibility, type safety, modularity and the availability of structured output enforcement for the AI integration layer. Cross-platform support was a foundational requirement: whilst primary development and testing was conducted on Windows 11, the chosen stack ensures the application compiles and runs natively on macOS and Linux distributions without modification. This is operationally significant for a security assessment tool, as practitioners commonly operate from Linux-based environments such as Kali Linux for security tooling, whilst organisational reporting and management workflows frequently run on Windows or macOS. A single codebase serving all three platforms avoids the maintenance burden and behavioural inconsistency of platform-specific implementations.

Two architectural decisions warrant specific justification. First, the choice to spawn Nmap as an external child process rather than embedding a scanning library was driven by the need to use Nmap’s full scripting engine, including the `vulners` and `http-headers` NSE scripts, which are not available through wrapper libraries. This design accepts an external dependency in exchange for access to the complete Nmap capability set. Second, the decision to use a single LLM provider (Gemini) with a model fallback chain rather than a multi-provider abstraction layer was a deliberate scope constraint. A 4-model fallback chain (`gemini-2.5-flash` as primary, with three progressively smaller alternatives) provides resilience against individual model unavailability without introducing the complexity of cross-provider prompt normalisation. The trade-off is vendor dependency, which is acknowledged as a limitation in Section 3.4.

The development and testing environment consisted of a Windows 11 workstation with Nmap installed on the system PATH. The controlled localhost testbed was served through Node.js on port 8080. All scanning was conducted over the local network or against public authorised targets, with no traffic routed through external proxies or VPNs. Build verification was additionally performed on Kali Linux to confirm that the Electron packaging, Nmap child process spawning and file system persistence behaved consistently across platforms.